Compressed Queries on the SAT Equation

bit, rank, select, and counting oracles for arbitrary CNFs

Oscar Riveros
Santiago, RM, Chile
oscar.riveros@gmail.com

DeepSeek V4.1 Flash 1017 via Kilo Code
Research collaborator (AI)
Ariadna Project

Working manuscript — October 28, 2026

Abstract

The SAT Equation maps a CNF F over n variables to a 2^n-bit integer $S(F)$ whose k-th bit is 1 exactly when the assignment of position k falsifies F. This paper studies how to query $S(F)$ without materializing its 2^n bits, and how much of that machinery can be brought to bear on P versus NP. Point queries cost $O(m)$ clause evaluations; the aggregate queries rank, select, #SAT, and uniform sampling carry the difficulty. We prove that a disjoint cover of the forbidden region is exactly equivalent to a CNF equivalent to F with pairwise disjoint falsifying cubes (DF-CNF); that a size-K DSOP answers rank in $O(nK)$ and select in $O(n^2K)$ arithmetic operations without materializing the truth table; that $\mathrm{rank}_0(k)$ is the number of models of F with $\mathrm{pos}(x) < k$, so any exact counter yields a rank oracle and, by binary search, select and uniform sampling; and that a hybrid algorithm uses the DSOP when it is small and the counter otherwise. The conceptual contribution is not the dense encoding but the barrier map: a uniform polynomial compiler for any of these representations would imply $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda's theorem, $\mathsf{PH} = \mathsf{P}$. The paper corrects the affine-query theorem (incompatible intersections contribute zero, not the volume of the piece), incorporates the known Sasao example, as discussed by Bernasconi, Ciriani, Luccio, and Pagli, and shows that it rules out DSOP as a universal route. It also states an explicit research program toward $\mathsf{P} = \mathsf{NP}$: lower bounds for ADSOP, affine backdoors, direct ZDD and Fourier compilation, and certificate verification. The subexponential instantiation with the OpenAI and Anthropic manuscripts (2026) is kept as hypothesis H-ETH and proved conditionally; it is not used as an established result. The experimental evaluation, still preliminary, separates compilation cost from query cost and confirms that the compiler, not the query engine, is the bottleneck.

Keywords: SAT Equation; COVERTRACE; knowledge compilation; disjoint subcubes; rank/select; ZDD; model counting.

Contents

1 Introduction **3**
1.1 Contributions 4
1.2 Organization 5

2 Preliminaries **5**
2.1 The Boolean cube and CNFs 5
2.2 The generalized SAT Equation 6
2.3 DSOP and DF-CNF 7
2.4 Queries over the bitvector 7
2.5 Compilers, certificates, and uniformity 7

2.6 Complexity model 8

3 Point queries **8**

4 The generalized form and DF-CNF **9**
4.1 The SAT Equation for arbitrary CNFs 9
4.2 DF-CNF: the exact duality 9

5 Rank and select from a DSOP **11**
5.1 Prefix counting in a cube 11
5.2 The DSOP engine 11
5.3 From CNF to DSOP 12

6 A counting oracle for rank and select **12**
6.1 The prefix comparator 12
6.2 Reduction from rank to exact counting 13
6.3 The H-ETH instantiation 13

7 The complexity frontier: barrier map and program toward $\mathsf{P} = \mathsf{NP}$ **13**
7.1 The collapse condition 14
7.2 The DSOP route: the certificate and its size 14
7.3 The counting route: $\#\mathsf{SAT}$ and the direction of the reduction 15
7.4 The H-ETH route: subexponential is not polynomial 15
7.5 The Fourier route 15
7.6 The ZDD route and non-uniformity 16
7.7 The stochastic route and the affine route 16
7.8 Summary of routes 16
7.9 Certificates, verification, and proof complexity 17
7.10 Islands of tractability and the coverage question 18
7.11 The islands are not an empty classification 18
7.12 Parallelization and the cost model 19
7.13 What would be needed for a collapse 20
7.14 Program toward $\mathsf{P} = \mathsf{NP}$ 20

8 Hybrid algorithm **21**
8.1 What the cap $K_{\max}$ guarantees 22
8.2 Why the hybrid is not inevitably exponential 23
8.3 Success conditions per branch 23

9 Optimization via ZDD and Fourier **25**
9.1 Fourier representation 25

10 Stochastic oracles and affine covers **26**
10.1 Oracle types and separations 26
10.2 Sampling, hashing, and Karp–Luby cancellation 27
10.3 Affine cover: ADSOP 29

10.4 Direct ADSOP compiler 32
10.5 Limits and open problems 35

11 Experimental protocol **36**
11.1 Instances 36
11.2 Metrics 36
11.3 Acceptance criteria 36
11.4 Experimental hypotheses 36
11.5 Protocol for the alternatives 36
11.6 Protocol for the hybrid 37
11.7 Three measurement scenarios 38
11.8 Preliminary results: DSOP versus Kerberos 38

12 Related work **40**

13 Discussion and open problems **41**
13.1 Unconditional results 41
13.2 The structural limit 42
13.3 Open problems 42
13.4 Alternatives and guarantees 43
13.5 Relation to practice 43

14 Conclusions **43**

A The H-ETH instantiation **44**

1 Introduction

The SAT Equation, introduced in [1] for balanced CNFs and generalized in [2] to arbitrary CNFs, encodes the truth table of a formula in a single integer. For a CNF F over n variables, $S(F)$ has 2^n bits; the bit at position $\mathrm{pos}(x)$ is 1 if and only if x falsifies F. The unifying note [4] proved that this integer and the COVERTRACE compilation of [3] represent the same object: the forbidden region $U(F) \subseteq \Omega_n$.

The representation is exact, but materializing it costs 2^n bits. The question of this paper is how to query it without constructing it. Two notions should be kept apart. A *point query* decides the bit $\mathrm{bit}(k)$ at a given position; for the SAT Equation it is direct, because $\mathrm{bit}(k) = 1 - F(\mathrm{assign}(k))$, where $F(x) \in \{0,1\}$ is the truth value of F at x. It costs $O(m)$ clause evaluations, the cost of evaluating the CNF. *Aggregate queries* count ones or zeros in a prefix (rank), locate the r-th one or zero (select), count the total (#SAT), or sample uniformly. They are not answered by evaluating a single assignment; they are the queries that encode the difficulty of model counting.

The thesis of the paper is that the generalized form of the SAT Equation is not only a dense representation of the truth table, but the natural interface for compiling query structures over it. The COVERTRACE DSOP provides a sparse representation that, when small, answers rank and select in time proportional to its size. When the DSOP is large, an exact model counter answers rank through the prefix restriction $\mathrm{pos}(x) < k$. The result is a hybrid engine that does not materialize $S(F)$ and uses each representation in the regime where it is competitive.

The paper pursues two goals that should not be confused. The first is technical: to build and analyze the query layer. The second is strategic: to determine which part of that layer can contribute to a proof of $\mathsf{P} = \mathsf{NP}$ and which part cannot. The short answer is that the query layer is not the problem; the problem is compilation. The paper turns that observation into a barrier map and a research program, not into a proof of collapse.

The paper also explores two alternatives to classical cube compilation. The first is stochastic: direct sampling, XOR hashing, and the Karp–Luby estimate of the forbidden DNF. The second is algebraic: a cover by affine subspaces over $\mathbb{F}_2$, which contains parity in a single piece and admits exact counting, ranking, and selection. Section 10 presents both, proves the correctness of affine queries, and describes a direct ADSOP compiler prototype that uses a SAT solver as an oracle.

The paper separates two levels of result. The theorems of Sections 4–6 are unconditional —the DSOP–DF-CNF equivalence, the rank–select engine, and the reduction from rank to exact counting—, with the exception of the subexponential instantiation in Appendix A, which is conditional on H-ETH. Section 7 analyzes the most delicate consequence: why the engine does not imply $\mathsf{P} = \mathsf{NP}$. The reason is structural: there is a separation between the cost of querying a representation and the cost of constructing it, and the paper proves exactly which additional condition would be needed for a collapse.

There is also a result that changes the map and that the paper incorporates as its own piece: the DSOP route cannot be universal. Sasao [25] exhibited a function with a SOP of $n/2$ terms whose DSOP has $2^{n/2}-1$ cubes; Bernasconi, Ciriani, Luccio, and Pagli [26] discuss that example as evidence that exponential growth persists for minimal solutions. Dualizing the construction yields a CNF with $O(n)$ clauses and $\chi_{\sqcup}(U(F)) = 2^{\Omega(n)}$ (Proposition 4.8). This does not prove $\mathsf{P} \neq \mathsf{NP}$; it proves that a proof of $\mathsf{P} = \mathsf{NP}$ cannot go through disjoint cubes. It is the first step of the program: close routes before opening others.[1]

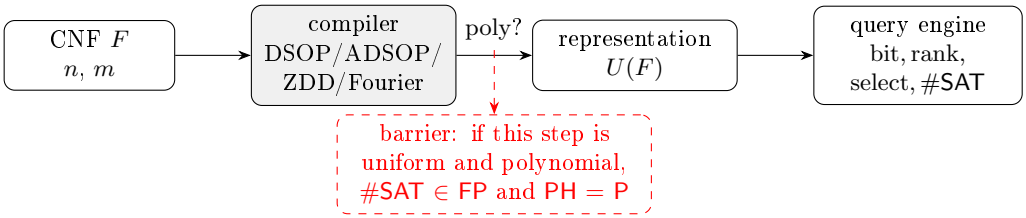


Figure 1: The engine separates the compilation phase from the query phase. The complexity difficulty is in the first; the second is what the paper solves.

1.1 Contributions

C1. Query model. We formalize bit, rank, select, #SAT, and uniform sampling over the bitvector $S(F)$ (Section 2).

C2. Generalized equation and DF-CNF. We prove that a disjoint cover of $U(F)$ is exactly equivalent to a CNF equivalent to F whose falsifying cubes are pairwise disjoint. This turns the carry–disjunction duality of [4] into an equality of minima and justifies that the output of COVERTRACE is a DF-CNF (Section 4).

C3. Rank/select from a DSOP. If $\mathcal{U}$ is a disjoint cover of size K, then $\mathrm{rank}_1(k)$ is computed in $O(nK)$ arithmetic operations, $\mathrm{select}_0(r)$ in $O(n^2K)$, #SAT in $O(K)$, and uniform sampling in $O(n^2K)$ (Section 5).

C4. Counting oracle. We prove that $\mathrm{rank}_0(k) = \#\mathsf{SAT}(F \wedge [\mathrm{pos}(x) < k])$. Any exact model counter yields a rank oracle; $O(n)$ queries yield select and sampling. The instantiation with

[1] An earlier version of this manuscript treated the DSOP separation as an open problem. Reading [26] and the original source [25] forced us to correct that claim; the result is stated in Proposition 4.8.

the OpenAI [5] and Anthropic [6] manuscripts is stated as hypothesis H-ETH (Appendix A).

C5. Hybrid algorithm and experimental protocol. We combine DSOP and counting oracle, analyze the cost, and propose a reproducible protocol (Sections 8 and 11).

C6. Fourier representation. We formalize the Walsh–Hadamard spectrum of $S(F)$; a spectrum of size s answers rank in $O(sn)$ and select in $O(sn^2)$. Parity has $s = 2$ and DSOP 2^{n-1}; a wide clause has $s = 2^w$ and DSOP 1. Constructing a polynomial spectrum for every CNF is a route to collapse and an open problem (Section 9.1).

C7. Separation of the DSOP route. We incorporate the Sasao construction [25], discussed by Bernasconi, Ciriani, Luccio, and Pagli [26], and prove that there is a family of CNFs with $O(n)$ clauses and $\chi_{\sqcup}(U(F)) = 2^{\Omega(n)}$. The DSOP cannot be the universal representation; any proof of $\mathsf{P} = \mathsf{NP}$ must go beyond disjoint cubes (Proposition 4.8).

C8. Complexity frontier and barrier map. We prove that a uniform polynomial DSOP compiler would imply $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda's theorem, $\mathsf{PH} = \mathsf{P}$; the same implication extends to ADSOP, ZDD, Fourier spectrum, and a polynomial exact counter. The engine provides none of those pieces: its polynomial bounds are conditional on the certificate or the counter. We formalize uniform compilers and certificates, define islands of tractability, and state an explicit program toward $\mathsf{P} = \mathsf{NP}$ (Section 7).

C9. ADSOP correction and validation. We correct the formula for the intersection of a pattern with an affine piece: an incompatible intersection contributes zero, not the volume of the piece. The correction propagates to the proof, the algorithm, and the experimental protocol, which includes empty intersections, zero-dimensional intersections, and compatible full-rank systems (Sections 10.3 and 11).

C10. Stochastic and affine alternatives. We analyze sampling and hashing oracles, the cancellation of the complement in Karp–Luby, and an affine-subspace cover (ADSOP) that answers exact queries. We present a direct SAT-based ADSOP compiler and validate it for $n \leq 10$; it is not a polynomial compiler (Section 10).

1.2 Organization

Section 2 fixes notation, the query model, and the distinction between compiler and certificate. Section 3 treats point queries. Section 4 proves the DF-CNF equivalence and the separation of the DSOP route. Section 5 builds the rank–select engine from a DSOP. Section 6 reduces rank to exact counting. Section 7 presents the barrier map, the islands of tractability, and the program toward $\mathsf{P} = \mathsf{NP}$; it is the section that organizes the complexity discussion. Section 8 presents the hybrid algorithm. Section 9 discusses ZDD and Fourier optimizations. Section 10 explores stochastic oracles and affine covers. Section 11 proposes the experimental protocol. Sections 12 and 13 position the work, and Section 14 closes.

2 Preliminaries

2.1 The Boolean cube and CNFs

Let $\Omega_n = \{0,1\}^n$. For $x = (x_1, \ldots, x_n) \in \Omega_n$, the *position* of x is

$$\mathrm{pos}(x) = \sum_{i=1}^{n} x_i \, 2^{n-i} \in \{0, \ldots, 2^n - 1\}.$$

The map pos is a bijection; its inverse is denoted $\mathrm{assign}(k)$. The order induced by pos is lexicographic order with x_1 as the most significant bit. We set 1 = true and 0 = false. For $x \in \Omega_n$, $F(x) = 1$ if x satisfies F and $F(x) = 0$ if it falsifies F.

A *clause* C is a disjunction of literals; a CNF is $F = \bigwedge_{j=1}^{m} C_j$. We assume, without loss of generality, that tautological clauses have been removed: a tautology forbids no assignment. The empty clause is allowed and forbids every assignment.

For a clause C, the *pattern* $p(C) \in \{0, 1, \bullet\}^n$ is defined by

$$p(C)_i = \begin{cases} 0, & x_i \in C, \\ 1, & \neg x_i \in C, \\ \bullet, & \text{otherwise.} \end{cases}$$

The *subcube* induced by a pattern $u \in \{0, 1, \bullet\}^n$ is

$$Q(u) = \{x \in \Omega_n : \; u_i \neq \bullet \Rightarrow x_i = u_i\}.$$

Its *support* is $\mathrm{supp}(u) = \{i : u_i \neq \bullet\}$, its *width* is $k(u) = |\mathrm{supp}(u)|$, and its *volume* is $\mathrm{vol}(u) = |Q(u)| = 2^{n-k(u)}$.

The *forbidden region* of F is

$$U(F) = \bigcup_{C \in F} Q(p(C)).$$

Then

$$\#\mathsf{SAT}(F) = 2^n - |U(F)|.$$

For an integer $y \geq 0$, $\mathrm{popcnt}(y)$ denotes the number of ones in its binary expansion.

2.2 The generalized SAT Equation

For a clause C, let $P_C = \mathrm{supp}(p(C))$ and $M_C = [n] \setminus P_C$. The *base contribution* is

$$B(C) = \sum_{i \in P_C} p(C)_i \, 2^{n-i}.$$

The *clause mask* is the integer

$$S_C = \bigvee_{M' \subseteq M_C} 2^{B(C) + \sum_{i \in M'} 2^{n-i}},$$

where $\vee$ is bitwise OR. Equivalently,

$$S_C = 2^{B(C)} \prod_{i \in M_C} \left(1 + 2^{2^{n-i}}\right), \tag{1}$$

where the product is arithmetic and produces no carries because the exponents $\sum_{i \in M'} 2^{n-i}$ are distinct for distinct subsets $M' \subseteq M_C$.

The *SAT Equation integer* of F is

$$S(F) = \bigvee_{C \in F} S_C. \tag{2}$$

We write $S(F)$ with exactly 2^n bits (positions $0, \ldots, 2^n - 1$). The content of the SAT Equation is that the bit $\mathrm{pos}(x)$ of $S(F)$ is 1 if and only if $x \notin \mathsf{SAT}(F)$.

2.3 DSOP and DF-CNF

A family $\mathcal{U} \subseteq \{0,1,\bullet\}^n$ is *disjoint* if $Q(u) \cap Q(v) = \varnothing$ for $u \neq v$. A *DSOP* of $S \subseteq \Omega_n$ is a disjoint family $\mathcal{U}$ with $\mathrm{Cov}(\mathcal{U}) = S$, where $\mathrm{Cov}(\mathcal{U}) = \bigcup_{u \in \mathcal{U}} Q(u)$. The *minimum size of a disjoint cover* is

$$\chi_{\sqcup}(S) = \min\{|\mathcal{U}| : \mathcal{U} \text{ disjoint and } \mathrm{Cov}(\mathcal{U}) = S\}.$$

A *DF-CNF* is a CNF $\mathcal{G} = \bigwedge_{j=1}^{K} D_j$ whose clauses are not tautological and whose falsifying cubes $Q(p(D_j))$ are pairwise disjoint.[2]

2.4 Queries over the bitvector

We fix the CNF F and its bitvector $S = S(F)$ of length 2^n, with $S[k] = \mathrm{bit}(k)$. We define:

$$\begin{aligned} \mathrm{bit}(k) &= S[k], \\ \mathrm{rank}_1(k) &= \sum_{j=0}^{k-1} S[j], & \mathrm{rank}_0(k) &= k - \mathrm{rank}_1(k), \\ \mathrm{select}_1(r) &= \min\{k : \mathrm{rank}_1(k+1) \geq r\}, & \mathrm{select}_0(r) &= \min\{k : \mathrm{rank}_0(k+1) \geq r\}, \end{aligned}$$

with $0 \leq k \leq 2^n$ and $1 \leq r \leq \mathrm{rank}_1(2^n)$ (respectively, $r \leq \mathrm{rank}_0(2^n)$). Moreover,

$$\#\mathsf{SAT}(F) = \mathrm{rank}_0(2^n) = 2^n - \mathrm{popcnt}(S).$$

A *uniform sample* returns $\mathrm{assign}(k)$ with $k = \mathrm{select}_0(R)$, where R is uniform in $\{1, \ldots, \mathrm{rank}_0(2^n)\}$ (or the analogous construction with ones).

Observation 2.1 (Convention)**.** The balanced half of [1] uses the convention 0 = true, 1 = false; the generalization of [2] and this paper use the standard convention 1 = true. The two encodings are complementary in position; the note [4] details the equivalence. Here the bit 1 always marks an assignment that falsifies F.

2.5 Compilers, certificates, and uniformity

To speak about collapse without ambiguity, we fix what it means to *produce* a representation. A *representation language* $\mathcal{R}$ is a set of finite descriptions: lists of patterns for DSOP, lists of affine pieces for ADSOP, diagrams for ZDD, lists of coefficients for Fourier, or programs for a counter. A *compiler* for $\mathcal{R}$ is a deterministic Turing machine M that, on a CNF F with n variables and m clauses, halts with a description $w \in \mathcal{R}$ such that, when w is a list of patterns, $\mathrm{Cov}(w) = U(F)$; in the case of a counter, w is a program that returns $\#\mathsf{SAT}(F)$. The compiler is *uniform and polynomial* if its running time and the length of w are bounded by $\mathrm{poly}(n, m)$.

A *certificate* for F in $\mathcal{R}$ is a string w together with a verification that w describes $U(F)$; the verifier is a polynomial-time algorithm with access to F and w. The existence of a polynomial-size certificate is a non-uniform statement: it says that the compact representation exists, not that it can be found. The difference is the same as the difference between NP and P, and it is the reason Theorem 7.1 requires a compiler and not only a certificate. In what follows, a "route" means one of these representations together with the question of its uniform compiler.

[2] The acronym DF stands for *disjoint falsifiers*. It should not be confused with the disjoint form of Darwiche–Marquis, which imposes a condition of disjoint supports between the parts of a conjunction; here the disjointness is between falsifying cubes. In the knowledge-compilation literature the dual form is called a *disjoint DNF* or *partitioned DNF* [9]; we use DF-CNF to emphasize that the input is a CNF and that the object is the dual form of a DSOP.

Definition 2.2 (Island of tractability)**.** A family $\mathcal{F}$ of CNFs is *tractable by* $\mathcal{R}$ if there is a uniform algorithm that, for every $F \in \mathcal{F}$ with n variables and m clauses, runs in time $\mathrm{poly}(n, m)$ and produces a description $w \in \mathcal{R}$ of $U(F)$ of size $\mathrm{poly}(n, m)$; the family is *tractable by counting* if there is an exact counter that runs in time $\mathrm{poly}(n, m)$ for every $F \in \mathcal{F}$.

Definition 2.2 is the unit of the program of Section 7: instead of asking whether *all* CNFs fall into one representation, we ask which families fall into it and whether their union covers the space. The answer for DSOP is already negative (Proposition 4.8); for the other representations it is the central open problem.

2.6 Complexity model

We use the RAM model with words of $\Theta(\log n)$ bits. The 2^n-bit integers are not materialized in the compressed algorithms. The rank and select counters have $O(n)$ bits; to isolate the combinatorics, we count each arithmetic operation on them as unit cost. In the bit model, each costs $O(n/\log n)$ word operations, so the time bounds are multiplied by that factor. Evaluating a CNF counts as $O(m)$ clause evaluations, or $O(nm)$ bit operations. The parameters are n (variables), m (clauses), and K (number of DSOP cubes). We distinguish *preprocessing* from *query*: the bounds of Sections 5 and 6 refer to a query once the structure is built or the counter is available. The only unproved hypothesis in this paper is H-ETH, stated explicitly in Appendix A; two recent manuscripts announce it, but we do not use it as established.

3 Point queries

Proposition 3.1 (The bit is direct evaluation)**.** *For every* $k \in \{0, \ldots, 2^n - 1\}$*,*

$$\mathrm{bit}(k) = 1 - F(\mathrm{assign}(k)).$$

In particular, $\mathrm{bit}(k)$ *is computed with* $O(m)$ *clause evaluations, or* $O(nm)$ *bit operations, by evaluating the* m *clauses of* F *at* $\mathrm{assign}(k)$*.*

Proof. By the definition of $S(F)$, the bit $\mathrm{pos}(x)$ is 1 if and only if x falsifies F. Taking $x = \mathrm{assign}(k)$ gives the identity. Evaluating a CNF with m clauses and n variables costs $O(nm)$ in the bit model, and $O(m)$ if each clause is counted as an operation on an n-bit pattern; we use the $O(m)$ bound for the number of clause evaluations. □

Observation 3.2 (Point access and aggregate access)**.** For π, a BBP-style algorithm computes a distant digit without computing the preceding ones because π is an analytic constant with a fast series. For $S(F)$, the bit $\mathrm{pos}(x)$ is the evaluation of F at x: there is no analytic unknown to discover. The difficulty is not in point access but in aggregate access: rank, select, #SAT, and sampling.

Proposition 3.3 (Block query)**.** *A block of* 2^r *consecutive bits of* $S(F)$ *is computed with* $O(2^r m)$ *clause evaluations by evaluating* F *at the* 2^r *corresponding assignments. If a DSOP of size* K *is available, the same block is computed in* $O(2^r nK)$ *operations without evaluating* F*.*

Proof. Immediate from Proposition 3.1 and from membership in a cube. □

Proposition 3.3 shows that a sufficiently large block query can be cheaper with the DSOP than with the CNF, but it does not solve the problem of counting a prefix of length 2^n. That is the object of Sections 5 and 6.

4 The generalized form and DF-CNF

4.1 The SAT Equation for arbitrary CNFs

Theorem 4.1 (Generalized SAT Equation; [2])**.** *Let F be a CNF over n variables and let $S(F)$ be the integer defined in (2). For every $x \in \Omega_n$, the bit $\mathrm{pos}(x)$ of $S(F)$ is 1 if and only if x falsifies F. In particular, $S(F)$ encodes exactly the truth table of F and*

$$\mathrm{popcnt}(S(F)) = |U(F)|, \qquad \#\mathsf{SAT}(F) = 2^n - \mathrm{popcnt}(S(F)).$$

Proof. For a clause C, equation (1) expands the product as the sum of $2^{|M_C|}$ powers of two with exponents

$$B(C) + \sum_{i \in M'} 2^{n-i}, \qquad M' \subseteq M_C.$$

The exponents are distinct: if two subsets M', M'' produced the same exponent, equality of the sums of powers of two 2^{n-i} would force $M' = M''$, by uniqueness of the binary expansion. Therefore the binary expansion of S_C has a 1 exactly at the positions of $Q(p(C))$, that is, at the assignments that falsify C. The OR in (2) is the union of these position sets, and bitwise OR is idempotent, so there are no carries. Thus the bit $\mathrm{pos}(x)$ of $S(F)$ is 1 if and only if x belongs to some $Q(p(C))$, that is, if and only if x falsifies some clause of F. The counting identities follow because the Hamming weight counts the positions with 1. □

Observation 4.2 (The balanced case)**.** If every clause has width n, then $M_C = \varnothing$ and $S_C = 2^{B(C)}$; arithmetic sum coincides with OR when clauses do not share a falsifying assignment. The generalized form replaces sum by OR to absorb overlaps and allows arbitrary widths. The balanced form is therefore the particular case with no free variables.

4.2 DF-CNF: the exact duality

Theorem 4.3 (DSOP $\leftrightarrow$ DF-CNF)**.** *For every CNF F over n variables,*

$$\chi_{\sqcup}(U(F)) = \min\{K : \exists\, \mathcal{G} \text{ DF-CNF with } K \text{ clauses and } \mathcal{G} \equiv F\}.$$

Moreover, if $\mathcal{U} = \{u_1, \ldots, u_K\}$ is a disjoint cover of $U(F)$, the CNF

$$\mathcal{G}_{\mathcal{U}} = \bigwedge_{j=1}^{K} C_{u_j}, \qquad C_u = \Big(\bigvee_{i:u_i=0} x_i \Big) \vee \Big(\bigvee_{i:u_i=1} \neg x_i \Big),$$

is a DF-CNF equivalent to F with $Q(p(C_{u_j})) = Q(u_j)$.

Proof. For a pattern u, the clause C_u contains exactly the literals that become false at u: if $u_i = 0$, the literal x_i is false when $x_i = 0$; if $u_i = 1$, the literal $\neg x_i$ is false when $x_i = 1$; if $u_i = \bullet$, there is no literal. Therefore $Q(p(C_u)) = Q(u)$.

If $\mathcal{U}$ is disjoint and covers $U(F)$, then

$$\mathsf{SAT}(\mathcal{G}_{\mathcal{U}}) = \bigcap_{u \in \mathcal{U}} (\Omega_n \setminus Q(u)) = \Omega_n \setminus \bigcup_{u \in \mathcal{U}} Q(u) = \Omega_n \setminus U(F) = \mathsf{SAT}(F).$$

Hence $\mathcal{G}_{\mathcal{U}} \equiv F$ and its falsifying cubes are the disjoint $Q(u)$. Conversely, if $\mathcal{G} = \bigwedge D_j$ is a DF-CNF equivalent to F, then $U(\mathcal{G}) = U(F)$ and the $Q(p(D_j))$ form a disjoint cover of $U(F)$ of size K. Therefore the minimum over DF-CNFs is at least $\chi_{\sqcup}(U(F))$; the construction above gives the other inequality. The extreme cases $U(F) = \varnothing$ and $U(F) = \Omega_n$ are covered by $K = 0$ (empty CNF) and $K = 1$ (empty clause), respectively. □

Corollary 4.4 (Zero carry and COVERTRACE)**.** *Let $\mathcal{U}$ be a disjoint cover of $U(F)$ of size K. Then*

$$S(F) = \sum_{u \in \mathcal{U}} S_{C_u}$$

as an arithmetic sum, without carries, and the number of clauses of the DF-CNF $\mathcal{G}_{\mathcal{U}}$ is K. In particular, the output of COVERTRACE [3] is a DF-CNF equivalent to F.

Proof. The supports of the masks S_{C_u} are the disjoint cubes $Q(u)$; therefore arithmetic sum and OR coincide. The second part is Theorem 4.3. □

Example 4.5 (Wide clause)**.** Let C be a clause of width w. Its forbidden region is a single cube of volume 2^{n-w}; the DSOP has $K = 1$ cube and the DF-CNF has one clause. The balanced form would require 2^{n-w} singletons to represent the same cube. This is why the generalized form is the right one for rank and select.

Example 4.6 (Parity)**.** Let $O_n = \{x : \bigoplus_i x_i = 1\}$ and let F_n be the CNF with one clause C_a for each $a \in O_n$, constructed so that $Q(p(C_a)) = \{a\}$. Then $U(F_n) = O_n$, $\chi_{\sqcup}(O_n) = 2^{n-1}$, and $S(F_n)$ is the Thue–Morse integer of [4], with $\mathrm{popcnt}(S(F_n)) = 2^{n-1}$. The bound is tight: any cube with at least one free coordinate contains assignments of both parities, so a disjoint cover of O_n uses only singletons; there are 2^{n-1}. The CNF F_n has 2^{n-1} clauses and therefore this example does not separate polynomial-size CNF from polynomial-size DSOP: the input grows with the representation. The real separation is given by the Sasao construction, presented next.

Lemma 4.7 (Sasao; recorded by Bernasconi et al.)**.** *There is a sequence of Boolean functions G_n over $N = O(n)$ variables, defined for even n, with a SOP of $n/2$ terms and a minimum DSOP of $2^{n/2} - 1$ cubes.*

Proof. The construction is due to Sasao [25], pp. 8:11–8:20. Bernasconi, Ciriani, Luccio, and Pagli [26], Introduction, p. 2 of arXiv:1204.5306, record it and present it as a case where exponential growth appears even for minimal solutions. We do not reproduce the minimality proof here; the lemma is used as a cited lower bound. □

Proposition 4.8 (DSOP is not universal)**.** *There is a family $\{F_n\}$ of CNFs with $N = O(n)$ variables and $O(n)$ clauses such that*

$$\chi_{\sqcup}(U(F_n)) = 2^{\Omega(n)}.$$

Proof. By Lemma 4.7, let $G_n = T_1 \vee \cdots \vee T_{n/2}$ over $N = O(n)$ variables, where each T_i is a conjunction of literals, and let

$$F_n = \bigwedge_{i=1}^{n/2} \neg T_i.$$

Each $\neg T_i$ is a clause, so F_n has $n/2$ clauses and $N = O(n)$ variables. Moreover, $F_n(x) = 0$ if and only if some $T_i(x) = 1$, that is, $U(F_n) = \{x : G_n(x) = 1\}$. Therefore a disjoint cover of $U(F_n)$ is exactly a DSOP of G_n, and $\chi_{\sqcup}(U(F_n)) = 2^{n/2} - 1 = 2^{\Omega(n)}$. □

Observation 4.9 (What the separation says and does not say)**.** Proposition 4.8 is an unconditional lower bound for the language of disjoint cubes. It is not a lower bound for SAT or for #SAT: the CNF F_n may still be easy for another representation, for example an ADSOP, a ZDD, or a specialized counter. What it rules out is the hope that a uniform polynomial DSOP compiler covers every CNF; that route, by itself, cannot give $\mathsf{P} = \mathsf{NP}$. The program of Section 7 consists in determining which representation, if any, occupies the place that DSOP leaves empty.

5 Rank and select from a DSOP

5.1 Prefix counting in a cube

For a pattern u and an integer $0 \le k \le 2^n$, let

$$\mathrm{pc}(u,k) = \big|Q(u) \cap \{x : \mathrm{pos}(x) < k\}\big|.$$

We write k in binary with n bits, $k = (k_1, \dots, k_n)$, and define the number of free coordinates of u in positions $i, \dots, n$ as

$$f_u(i) = \big|\{j \ge i : u_j = \bullet\}\big|.$$

We set $f_u(n+1) = 0$.

Lemma 5.1 (Prefix of a cube)**.** *For every pattern u and every $0 \le k \le 2^n$, $\mathrm{pc}(u,k)$ is computed in $O(n)$ arithmetic operations and $O(n)$ additional bits. For $k = 2^n$, $\mathrm{pc}(u, 2^n) = \mathrm{vol}(u)$.*

Proof. For $k = 2^n$ the identity is immediate. For $0 \le k < 2^n$, write $k = (k_1, \dots, k_n)$ and define C_i as the number of assignments in $Q(u)$ that agree with k on the first $i-1$ positions and are strictly smaller at position i or at some later position, with $C_{n+1} = 0$. The recurrence is

$$C_i = \begin{cases} 2^{f_u(i+1)}, & u_i \neq \bullet \text{ and } u_i < k_i, \\ 0, & u_i \neq \bullet \text{ and } u_i > k_i, \\ C_{i+1}, & u_i \neq \bullet \text{ and } u_i = k_i, \\ C_{i+1}, & u_i = \bullet \text{ and } k_i = 0, \\ 2^{f_u(i+1)} + C_{i+1}, & u_i = \bullet \text{ and } k_i = 1. \end{cases}$$

The recurrence traverses exactly the options for bit i: those that already make the prefix smaller contribute all their extensions, $2^{f_u(i+1)}$; the one that agrees with k_i continues; the larger ones do not contribute. By descending induction on i, C_i counts the assignments in $Q(u)$ that agree with k on the first $i-1$ positions and are smaller at position i or at some later position, and C_1 is $\mathrm{pc}(u,k)$. The traversal has n steps; the counter and the powers $2^{f_u(i)}$ occupy $O(n)$ bits. □

Observation 5.2. The procedure above is a two-state automaton over the bits of k; the *tight* flag is the only necessary state. The cube is never materialized.

5.2 The DSOP engine

Theorem 5.3 (Rank/select with a DSOP)**.** *Let $\mathcal{U}$ be a disjoint cover of $U(F)$ of size K. Then:*

(a) $\mathrm{rank}_1(k) = \sum_{u \in \mathcal{U}} \mathrm{pc}(u,k)$ *and* $\mathrm{rank}_0(k) = k - \mathrm{rank}_1(k)$ *are computed in $O(nK)$ arithmetic operations.*

(b) $\#\mathsf{SAT}(F) = 2^n - \sum_{u \in \mathcal{U}} \mathrm{vol}(u)$ *is computed in $O(K)$ arithmetic operations.*

(c) $\mathrm{select}_0(r)$ *and* $\mathrm{select}_1(r)$ *are computed in $O(n^2 K)$ arithmetic operations using binary search over* rank_0 *or* rank_1*.*

(d) If $\mathrm{rank}_0(2^n) > 0$*, a uniform zero is obtained in $O(n^2 K)$ arithmetic operations by choosing R uniform in $\{1, \dots, \mathrm{rank}_0(2^n)\}$ and returning* $\mathrm{select}_0(R)$*. The analogous statement holds for ones.*

Proof. (a) Disjointness of $\mathcal{U}$ implies

$$\{x : \mathrm{pos}(x) < k\} \cap U(F) = \bigsqcup_{u \in \mathcal{U}} \big(\{x : \mathrm{pos}(x) < k\} \cap Q(u)\big),$$

and the cardinalities add. Each term is $\mathrm{pc}(u,k)$, computable in $O(n)$ by Lemma 5.1. The identity for rank_0 is immediate.

(b) $\#\mathsf{SAT}(F) = 2^n - |U(F)|$ and $|U(F)| = \sum_{u \in \mathcal{U}} \mathrm{vol}(u)$ by disjointness.

(c) The function $k \mapsto \mathrm{rank}_0(k)$ is nondecreasing and $\mathrm{rank}_0(2^n) = \#\mathsf{SAT}(F)$. Binary search over $k \in \{0, \ldots, 2^n - 1\}$ uses $O(n)$ queries to rank_0; each query costs $O(nK)$ by (a), hence $O(n^2K)$. The same argument holds for select_1 with rank_1.

(d) If R is uniform in $\{1, \ldots, \mathrm{rank}_0(2^n)\}$, the map $R \mapsto \mathrm{select}_0(R)$ is a bijection between that set and the positions of the zeros, because select_0 is strictly increasing. Therefore the returned zero is uniform. □

Corollary 5.4 (Polynomial bound, given the certificate)**.** *Let $\mathcal{U}$ be a disjoint cover of $U(F)$ of size $K = \mathrm{poly}(n, m)$. Then, given $\mathcal{U}$, the queries* bit, $\#\mathsf{SAT}$, rank, *and* select *are answered in polynomial time in n, m, K. If in addition $\mathcal{U}$ is constructed in polynomial time from F, then SAT and $\#\mathsf{SAT}$ for F are decided and counted in polynomial time; in particular, one can extract a satisfiability or falsification witness and verify it with one evaluation of F.*

Proof. Point evaluation is $O(m)$; the other bounds are those of Theorem 5.3. For a witness, use $\mathrm{select}_0(1)$ or $\mathrm{select}_1(1)$ and evaluate F at the resulting assignment. The second statement is the application of Theorem 5.3 to the already constructed cover. □

Observation 5.5 (The dominant cost is K)**.** Theorem 5.3 does not assume that K is small. If $K = 2^{n-1}$ (parity), the engine is exponential in n; if $K = 1$ (wide clause), every query is $O(n)$ or $O(n^2)$. The practical question is whether the DSOP of a CNF family is small; that is exactly the question of knowledge compilation [9, 3]. For parity, the input CNF also has 2^{n-1} clauses; the example shows the limit of the cube language, but it is not the separation between polynomial-size CNF and polynomial-size DSOP. That separation exists and is given by another family: Proposition 4.8, with $O(n)$ clauses and $K = 2^{\Omega(n)}$.

Example 5.6 (Wide clause, explicit computation)**.** Let $n = 4$ and $C = (x_1 \vee x_2)$, of width $w = 2$. Then $U(C) = \{0000, 0001, 0010, 0011\}$ and $S(C) = 1 + 2 + 4 + 8 = 15$. The DSOP has $K = 1$ cube $Q(00\bullet\bullet)$ (with $x_1 = x_2 = 0$), of volume 4. For $k = 3$, $\mathrm{pc}(u, 3) = 3$ because positions $0, 1, 2$ are in the cube and position 3 is not smaller than 3. For $k = 4$, $\mathrm{pc}(u, 4) = 4$. The query is $O(n)$.

5.3 From CNF to DSOP

Theorem 5.3 assumes a disjoint cover $\mathcal{U}$. COVERTRACE [3] constructs one for $U(F)$ using *CubeDiff* / *AddCube*; its output size is K. Theorem 4.3 guarantees that this output is a DF-CNF equivalent to F. If K is small, queries are polynomial once $\mathcal{U}$ is built. The cost of building it is that of COVERTRACE: its bound is $O((T + m)c(n))$, where T counts the pieces produced with multiplicity; the bound is in T, not in the final size K. Parity shows that K can be 2^{n-1}; Proposition 4.8 shows that there are families with $K = 2^{\Omega(n)}$ and only $O(n)$ clauses. Section 7 analyzes the scope of this bound and its relation to the input size.

6 A counting oracle for rank and select

6.1 The prefix comparator

Lemma 6.1 (Comparator CNF)**.** *For every n and every $0 \le k \le 2^n$ there is a CNF $\mathcal{C}_k$ with $O(n)$ variables and $O(n)$ clauses, over a variable set that includes $x_1, \ldots, x_n$, such that the Tseitin encoding of the comparator has a unique extension for each $x \in \Omega_n$, and $\mathcal{C}_k$ (with its output forced to 1) is satisfiable if and only if $\mathrm{pos}(x) < k$. For $k = 2^n$, the condition is empty.*

Proof. Construct a standard comparator of two n-bit numbers with a three-state automaton (less, equal, greater) that scans positions from left to right. Each transition is a Boolean gate of bounded fan-in, and the circuit has $O(n)$ gates. The Tseitin encoding of each gate with an auxiliary variable produces $O(n)$ clauses and forces a unique value for each auxiliary given x. Add the unit clause that fixes the "less" output to 1. Therefore each x has at most one satisfying extension, and that extension exists exactly when the comparator output is true, that is, when $\mathrm{pos}(x) < k$. For $k = 2^n$ use the empty CNF, since every x satisfies the condition. □

6.2 Reduction from rank to exact counting

Theorem 6.2 (Rank from an exact counter)**.** *Let A be an exact algorithm that, given a CNF with N variables and M clauses, returns its number of models in time $T(N, M)$ and space $S(N, M)$. Then, for every CNF F over n variables and every prefix $0 \le t \le 2^n$,*

$$\mathrm{rank}_0(t) = \#\mathsf{SAT}\big(F \wedge \mathcal{C}_t\big),$$

and $\mathrm{rank}_0(t)$ is computed in time $T(n + O(n), m + O(n)) + O(n)$ and space $S(n + O(n), m + O(n)) + O(n)$. Moreover, $\mathrm{select}_0(r)$ requires $O(n)$ calls to A and is computed in time $O\big(n \cdot (T(n + O(n), m + O(n)) + n)\big)$; uniform sampling requires $O(n)$ calls in total.

Proof. By Lemma 6.1, the auxiliary extensions of $\mathcal{C}_t$ are unique for each x. Therefore the number of models of $F \wedge \mathcal{C}_t$ over all variables is exactly the number of $x \in \mathsf{SAT}(F)$ with $\mathrm{pos}(x) < t$, which is $\mathrm{rank}_0(t)$ by definition. The time bound is that of A on the combined CNF, plus the cost of constructing the comparator. For select_0, binary search over t as in Theorem 5.3: $O(n)$ queries to rank_0; correctness follows from monotonicity of rank_0. For sampling, query $\mathrm{rank}_0(2^n)$, choose R uniform in $\{1, \ldots, \mathrm{rank}_0(2^n)\}$, and return $\mathrm{select}_0(R)$. □

Observation 6.3. Theorem 6.2 is valid for any exact counter: it does not depend on which algorithm is used. Its practical interest depends on A being faster than materializing $S(F)$ and on the combined CNF preserving a favorable structure, for example remaining k-CNF or having a DSOP that does not blow up.

6.3 The H-ETH instantiation

The subexponential instantiation with H-ETH is presented in Appendix A. It is a conditional note and may be skipped on a first reading: it does not affect the results of Sections 4–5 or the barrier map.

7 The complexity frontier: barrier map and program toward P = NP

The engine of Sections 5–6, together with the optimizations of Section 9, answers queries over $S(F)$ without materializing its 2^n bits. Since $\mathrm{rank}_0(2^n) = \#\mathsf{SAT}(F)$, a hasty reading could conclude that the engine decides SAT in polynomial time. The conclusion does not follow. The reason is a separation of costs: the query bounds are polynomial in n and in the size K of the certificate, but the certificate and the counter are not free. This section sharpens the separation, surveys the routes by which a collapse could be attempted, and turns the negative result into a research program. The question organizing the program is not "does the engine solve $\mathsf{P} = \mathsf{NP}$?" but "what would have to be proved for one of these routes to solve it, and what is already known about each?".

7.1 The collapse condition

To decide SAT it suffices to compute $\#\mathsf{SAT}(F)$ and compare it with zero. Therefore any algorithm that computes $\#\mathsf{SAT}$ for every CNF in polynomial time implies $\mathsf{P} = \mathsf{NP}$; in fact, it implies $\#\mathsf{P} \subseteq \mathsf{FP}$ and, by Toda's theorem [15], $\mathsf{PH} = \mathsf{P}$.

Theorem 7.1 (Collapse under uniform compilation)**.** *If there is a deterministic algorithm that, for every CNF F over n variables and m clauses, runs in time $\mathrm{poly}(n, m)$ and produces a disjoint family $\mathcal{U}$ with $\mathrm{Cov}(\mathcal{U}) = U(F)$ and $|\mathcal{U}| \leq \mathrm{poly}(n, m)$, then $\#\mathsf{SAT} \in \mathsf{FP}$ and $\mathsf{PH} = \mathsf{P}$. In particular, $\mathsf{P} = \mathsf{NP}$.*

Proof. Given F, the algorithm produces $\mathcal{U}$ in polynomial time. By disjointness,

$$|U(F)| = \sum_{u \in \mathcal{U}} 2^{n-k(u)},$$

which is computed in polynomial time. Therefore $\#\mathsf{SAT}(F) = 2^n - |U(F)| \in \mathsf{FP}$. Since $\#\mathsf{SAT}$ is $\#\mathsf{P}$-complete [14], $\#\mathsf{P} \subseteq \mathsf{FP}$. By Toda [15], $\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}} = \mathsf{P}$; since $\mathsf{P} \subseteq \mathsf{PH}$, we get $\mathsf{PH} = \mathsf{P}$ and, in particular, $\mathsf{P} = \mathsf{NP}$. □

Corollary 7.2 (The barrier in contrapositive form)**.** *If $\mathsf{PH} \neq \mathsf{P}$, then there is no uniform polynomial DSOP compiler with polynomial output for every CNF. The same statement holds for ADSOP, ZDD, Fourier spectrum, and a polynomial exact counter.*

Proof. This is the contrapositive of Theorem 7.1 and of the variants obtained by replacing the representation with any of the others; in every case, the polynomial-size representation allows $\#\mathsf{SAT}$ to be computed in polynomial time, and Toda's argument gives $\mathsf{PH} = \mathsf{P}$. □

The theorem is the same implication as Theorem 7.4 of [3]. Its hypothesis is strong: a uniform, polynomial compiler with polynomial output for every CNF. The engine of this paper is not that compiler. It takes a cover $\mathcal{U}$ as input; correctness and query cost are proved for an already constructed $\mathcal{U}$. The question, then, is whether any of the engine's routes provides the hypothesis of Theorem 7.1. In this paper the answer is no for DSOP, and remains open for the others: none of the routes is proved uniform and polynomial for every CNF, and each fails for a different reason.

7.2 The DSOP route: the certificate and its size

Theorem 5.3 is polynomial in n and K. If K is polynomial in the input size $n + m$, queries are polynomial in the input; if K is exponential in n but the input is also exponential, the cost can be exponential in n and polynomial in m. Parity illustrates the second case: $\chi_{\sqcup}(O_n) = 2^{n-1}$ (Example 4.6), but the parity CNF has 2^{n-1} clauses, so $K = 2^{n-1}$ is linear in m. That example, by itself, does not separate polynomial-size CNF from polynomial-size DSOP. The separation is given by the Sasao construction and Bernasconi et al.: Proposition 4.8 exhibits a family with $m = O(n)$ and $\chi_{\sqcup} = 2^{\Omega(n)}$. Therefore *there is no* uniform polynomial DSOP compiler for every CNF: the output cannot be polynomial in the input for that family. The DSOP route is closed as a universal route; it is not closed as an island of tractability. The DSOP–DF-CNF equivalence of Theorem 4.3 does not remove this difficulty: it is an exact rewriting of the same forbidden region, and the resulting DF-CNF has exactly K clauses; K remains the certificate.

The hypothesis of Theorem 7.1 is a uniform polynomial compiler in the input size. The paper does not provide it. COVERTRACE has a bound $O((T + m)c(n))$, where T counts the pieces produced with multiplicity; it is a parameterized bound, not a guarantee of polynomial time in $n + m$ when K grows. The cap $K_{\max}$ limits the output size; it does not turn COVERTRACE

into a polynomial compiler in the input size. A collapse would require a uniform polynomial compiler, and the paper does not provide one.

The correct consequence is local: if a family of CNFs admits a DSOP of size polynomial in the input and a polynomial compiler, then SAT and #SAT are polynomial on that family. It is a structural result of the same kind as the tractability of bounded-treewidth formulas, not a collapse of classes.

7.3 The counting route: #SAT and the direction of the reduction

Theorem 6.2 proves

$$\mathrm{rank}_0(t) = \#\mathsf{SAT}(F \wedge \mathcal{C}_t).$$

It is a polynomial reduction from rank to #SAT: it shows that rank is no harder than #SAT. It does not show that #SAT is easy. The direction matters. In the other direction, $\#\mathsf{SAT}(F) = \mathrm{rank}_0(2^n)$ is a single rank query; therefore rank and #SAT are polynomial-time equivalent. The rank oracle is exactly as hard as #SAT.

The prefix comparator is parsimonious: each x has a unique extension to the auxiliary variables, so the count is preserved. The encoding adds $O(n)$ variables and $O(n)$ clauses, but it does not simplify F; if F is hard for #SAT, so is $F \wedge \mathcal{C}_t$. The reduction serves to compute rank from a counter, not to build the counter.

If the counter A were polynomial for every CNF, it would follow that $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$. But A is an oracle; Theorem 6.2 is valid for any exact counter and does not assert that a polynomial one exists. Hypothesis H-ETH gives a subexponential counter, not a polynomial one. That difference is decisive and is analyzed next.

7.4 The H-ETH route: subexponential is not polynomial

Hypothesis H-ETH is stated in Appendix A; here only its consequence is used. H-ETH provides a counter in time $2^{o(n)}\,\mathrm{poly}(n, m)$, not in time $\mathrm{poly}(n, m)$. The distance between the two regimes is the object of P versus NP. A subexponential algorithm for #SAT refutes #ETH and hence ETH, but it does not imply $\mathsf{P} = \mathsf{NP}$.

The logical relation is precise. If $\mathsf{P} = \mathsf{NP}$, then 3SAT would have a polynomial algorithm and hence one of time $2^{o(n)}$, contradicting ETH. Thus $\mathsf{P} = \mathsf{NP}$ implies that ETH is false; equivalently, ETH implies $\mathsf{P} \neq \mathsf{NP}$. The negation of ETH is a necessary condition for $\mathsf{P} = \mathsf{NP}$, not a sufficient one. H-ETH, even if verified, would leave open whether the counter can be brought down to polynomial time. Corollary A.3 is a conditional subexponential result, not a proof of $\mathsf{P} = \mathsf{NP}$.

For CNFs of arbitrary width there is a second limitation: the encoding to k-CNF adds auxiliary variables and the resulting bound is in the number of encoded variables, not necessarily in the original n. The DSOP route does not have this width restriction; hence the two routes are complementary.

7.5 The Fourier route

If the Walsh–Hadamard spectrum of $S(F)$ has s nonzero coefficients and they are known, the Fourier engine of Section 9.1 answers rank in $O(sn)$ and select in $O(sn^2)$, and #SAT is $2^n - \hat{f}(\varnothing)$. Parity has $s = 2$, so Fourier compresses exactly what DSOP does not. But the spectrum can be dense: a wide clause of width w has $s = 2^w$, and a random function has a dense spectrum. The algorithm of [7] computes the dense transform of length 2^n in $O(2^n n^{1-\delta})$, still exponential in n; it is not a sparse-spectrum compiler. A spectrum of polynomial size constructed in polynomial time for every CNF would give $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$; the paper does not provide it.

The barrier is the same as in the other routes: the representation must be small and constructible in polynomial time.

7.6 The ZDD route and non-uniformity

A ZDD of size Z answers queries in $O(n)$, but Z can be exponential. If it is built from a DSOP of K cubes, $Z = O(nK)$; reduction can decrease it, as in parity, where $Z = O(n)$ and $K = 2^{n-1}$. The advantage of the ZDD is real, but it presupposes construction. Direct compilation from a CNF to a small ZDD, without going through the DSOP, is an open problem (Section 13). If there were a uniform algorithm that, for every CNF with n variables and m clauses, ran in time $\text{poly}(n, m)$ and produced a ZDD of size $\text{poly}(n, m)$, counting on the ZDD would give $\#\mathsf{SAT} \in \mathsf{FP}$ and, by the same argument, $\mathsf{PH} = \mathsf{P}$. The bound must be in n, m, not in the size of the ZDD: a polynomial algorithm in Z that produces an exponential ZDD is useless for the collapse. The paper does not provide that algorithm; the ZDD is an optimization conditional on the size of the structure.

There is a further distinction between certificate and algorithm. A polynomial-size DSOP or ZDD for every F is a non-uniform certificate: it says that the representation exists, not that it can be found in polynomial time. P versus NP is a question about uniform algorithms. A polynomial certificate per instance is not enough by itself to answer it; a uniform compiler is needed. Theorem 7.1 states that condition and shows that, if it held, the collapse would be even stronger: $\mathsf{PH} = \mathsf{P}$.

7.7 The stochastic route and the affine route

Section 10 adds two routes that do not appear in the engine of the previous sections. The stochastic one is not a route to collapse in the same sense: sampling and hashing give relative error, not exactness. An FPRAS for $\#\mathsf{SAT}$ for every CNF would imply $\mathsf{NP} \subseteq \mathsf{BPP}$: an instance with $\#\mathsf{SAT} = 0$ returns 0 and one with $\#\mathsf{SAT} > 0$ returns a positive value with probability at least $1 - \delta$, so SAT is decided in BPP. Karp–Luby gives an FPRAS for $|U(F)|$, but the subtraction $2^n - |U|$ does not preserve relative error. Thus randomness changes the guarantee from exact to approximate; it does not provide $\#\mathsf{SAT} \in \mathsf{FP}$.

The affine route is exact. ADSOP contains parity in one piece and answers counting, ranking, and selection in time polynomial in its size (Theorem 10.4). If there were a uniform polynomial ADSOP compiler for every CNF, it would follow that $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$. The prototype of Section 10.4 is not that compiler: it uses a SAT oracle, makes $O(n^2)$ queries per node, and may explore an exponential number of nodes. The DSOP parity bound is not inherited, but ADSOP has its own compilation barrier, which remains open.

7.8 Summary of routes

Table 1 summarizes the seven routes. The column "if polynomial" indicates the consequence of turning the route into a uniform polynomial algorithm for every CNF; the last column indicates what the paper provides.

Route	What is needed	If polynomial	What the paper provides
DSOP	uniform compiler producing $\mathcal{U}$ of size $\mathrm{poly}(n, m)$	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	polynomial queries given $\mathcal{U}$ (Theorem 5.3)
Counter	polynomial exact counter for every CNF	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	reduction rank $\leq \#\mathsf{SAT}$ (Theorem 6.2); H-ETH gives $2^{o(n)}$
ZDD	uniform polynomial construction of a polynomial ZDD	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	conditional optimization (Section 9)
Fourier	WHT spectrum of size $\mathrm{poly}(n, m)$ built in time $\mathrm{poly}(n, m)$	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	$O(sn)$ queries given the spectrum (Section 9.1)
ADSOP	uniform ADSOP compiler of size $\mathrm{poly}(n, m)$	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	exact queries given an ADSOP (Theorem 10.4); prototype (Section 10.4)
Stochastic	FPRAS for #SAT with relative error for every CNF	FPRAS for $\#\mathsf{SAT} \Rightarrow \mathsf{NP} \subseteq \mathsf{BPP}$ (approximation, not exactness)	sampling, hashing, and Karp–Luby (Section 10.2)
Bit	evaluation of F at an assignment	no collapse	$O(m)$ per query (Proposition 3.1)

Table 1: The engine's routes and the exact collapse condition. The DSOP route is ruled out as a universal route by Proposition 4.8; the others remain open.

7.9 Certificates, verification, and proof complexity

The distinction between certificate and algorithm deserves separate treatment because a DSOP looks like an NP certificate and yet is not one. A satisfiability certificate is an assignment verified in $O(m)$. A DSOP certificate is a list of cubes covering $U(F)$; verifying it requires checking that the union is exactly $U(F)$, and containment of a cube in a union of cubes is coNP-hard in general. The reason is direct: the complement of containment is satisfiability of the CNF formed by the complements of the cubes, restricted to the cube, and that problem is NP-complete. Therefore a DSOP certificate is not an NP certificate in the usual sense: verification is not polynomial without a construction trace.

This observation connects the parameter K with proof complexity. The size of a DSOP is the measure of a refutation in a natural proof system when F is unsatisfiable: in that case $U(F) = \Omega_n$, and a DSOP of Ω_n with K cubes is a refutation of size K in the disjoint-cube system. Comparing K with resolution, *cutting planes*, or *polynomial calculus* is a program in itself; the lower bounds of Ben-Sasson and Wigderson [33] and the proof-complexity literature [34] provide the framework. Proposition 4.8 is a lower bound of this kind for a concrete family, though not necessarily an unsatisfiable one: there are CNFs with $O(n)$ clauses whose forbidden region has a minimum disjoint cover of size $2^{\Omega(n)}$.

Open Problem (Coverage verification)**.** Determine whether a DSOP of $U(F)$ can be verified in polynomial time with a construction trace of polynomial size, and characterize the families for which verification without a trace is polynomial. Containment of a cube in a union of cubes is coNP-hard in general, so the certificate must include more than the list of cubes.

The consequence for the program is clear: a polynomial-size certificate for every F does not solve $\mathsf{P} = \mathsf{NP}$; what is needed is a *finder* of certificates. Proof complexity studies the existence and size of certificates; compilation studies their search. The engine of this paper is the interface between the two: given a representation, it answers queries; given a trace, it verifies it; what it does not do is find the representation.

7.10 Islands of tractability and the coverage question

Definition 2.2 and Table 1 allow the program to be stated without ambiguity. Each representation defines an island: the CNFs that admit a uniform polynomial compiler for that representation. The first observation is that all islands lie inside P.

Proposition 7.3 (The islands are in P)**.** *Let $\mathcal{F}$ be a family of CNFs tractable by DSOP, ADSOP, ZDD, Fourier spectrum, or counting, in the sense of Definition 2.2. Then SAT and $\#\mathsf{SAT}$ are solved in polynomial time for every $F \in \mathcal{F}$.*

Proof. By definition, there is a uniform polynomial compiler producing the representation. For DSOP, Theorem 5.3 gives $\#\mathsf{SAT}$ in $O(K)$ and rank in $O(nK)$; for ADSOP, Theorem 10.4 gives counting and queries; for ZDD, the number of sets is $|U(F)|$ and $\#\mathsf{SAT} = 2^n - |U(F)|$, computable by traversing the diagram; for Fourier, $\#\mathsf{SAT} = 2^n - \hat{f}(\varnothing)$ and the spectrum is assumed constructed; for counting, the counter is the algorithm itself. In every case, SAT is decided by comparing $\#\mathsf{SAT}$ with zero. The composition of a polynomial compiler with a polynomial query is polynomial. □

The question of P versus NP, in this language, has a sufficient form: that the union of the islands covers every CNF. Proposition 4.8 already gives a negative answer for the DSOP island. For the others the answer is unknown, and that is where the program begins. Two incompatible conjectures should be separated, because one of the two must be false.

Conjecture 7.4 (Affine universality, high risk)**.** Every CNF with n variables and m clauses admits an ADSOP of size $\mathrm{poly}(n, m)$ constructible in time $\mathrm{poly}(n, m)$. If this conjecture is true, then $\#\mathsf{SAT} \in \mathsf{FP}$ and $\mathsf{P} = \mathsf{NP}$.

Conjecture 7.5 (Affine separation)**.** There is a family of CNFs with $m = \mathrm{poly}(n)$ whose minimum ADSOP has size $2^{\Omega(n)}$. If this conjecture is true, the affine route is closed as a universal route, just like the DSOP route.

We have a proof of neither. The experimental evidence of Section 10.3 is ambiguous: ADSOP compresses parity and affine subspaces, but not a random 3-CNF. That evidence is compatible with affine separation and also with a universality that requires a construction smarter than the prototype. The value of the pair of conjectures is logical: they fix exactly what would have to be proved for the affine route to solve $\mathsf{P} = \mathsf{NP}$ and what would have to be proved to close it.

Conjecture 7.6 (Affine backdoor)**.** There is a family of 3-CNFs with $m = O(n)$ that admits no affine backdoor of size $O(\log n)$; in fact, random 3-CNFs near the threshold do not admit one with probability $1 - o(1)$.

Conjecture 7.6 is the quantitative version of the affine-backdoor route: if every CNF had a backdoor of size $O(\log n)$, counting would reduce to $\mathrm{poly}(n)$ linear systems and we would have a polynomial algorithm. The paper does not prove it; it states it because it turns a practical intuition into a falsifiable claim. The evidence from random 3-CNFs is indirect: the ADSOP prototype does not compress them, but that does not prove the nonexistence of a small backdoor.

7.11 The islands are not an empty classification

Table 3 shows that the islands contain nontrivial families. A wide clause is a DSOP of size one; parity is an ADSOP of size one and an exponential DSOP; a bounded-treewidth CNF falls into counting or ZDD; a CNF with a small affine backdoor falls into affine counting. What is not known is whether these islands, together, cover every CNF. The experimental evidence of Section 11 suggests that random 3-CNFs near the threshold fall outside the known islands, but an absence of experimental compression is not a lower bound.

7.12 Parallelization and the cost model

A natural objection is the following: if the engine's queries are parallelizable, can compilation or counting also be parallelized to obtain a polynomial algorithm for SAT? The objection confuses depth with work. A parallel algorithm that uses $T(N)$ time and $P(N)$ processors performs $W(N) = O(T(N)P(N))$ operations. Step-by-step sequential simulation executes, at each step, the operations of the $P(N)$ processors; it costs $O(W(N))$ time.

Proposition 7.7 (Sequential simulation)**.** *Let A be a uniform parallel algorithm that, for inputs of size N, uses time $T(N)$ and $P(N)$ processors. Then A is simulated sequentially in time $O(T(N)P(N))$. In particular, if $T(N)P(N) = \mathrm{poly}(N)$, the problem solved by A is in* P*; the* NC *case is $T(N) = \mathrm{polylog}(N)$ and $P(N) = \mathrm{poly}(N)$.*

Proof. The simulation traverses steps $t = 1, \ldots, T(N)$; at step t it sequentially executes the operations of the $P(N)$ processors on shared memory. Each operation is simulated in $O(1)$ time in the standard PRAM model, so the total is $O(T(N)P(N))$. If $T(N)P(N)$ is polynomial, the simulation is polynomial; the NC case is a particular case. This is the inclusion $\mathsf{NC} \subseteq \mathsf{P}$: NC is the class of problems admitting uniform circuits of polynomial size and polylogarithmic depth, equivalently uniform PRAM algorithms of polylogarithmic time and polynomial processors [19]. □

Corollary 7.8 (Full parallelization would imply collapse)**.** *If the complete engine —that is, DSOP compilation or the exact counter together with the query phase— were a uniform* NC *algorithm for every CNF, then $\#\mathsf{SAT} \in \mathsf{P}$ and $\mathsf{P} = \mathsf{NP}$.*

Proof. By Proposition 7.7, the sequential simulation of the NC algorithm is polynomial. The engine would compute #SAT in polynomial time for every CNF. Since #SAT is #P-complete [14], $\#\mathsf{P} \subseteq \mathsf{FP}$; by Toda [15], $\mathsf{PH} = \mathsf{P}$. In particular, SAT reduces to #SAT by comparison with zero, so $\mathsf{P} = \mathsf{NP}$. □

The corollary confirms the intuition: if *the whole* engine were parallelizable with polynomial work, the collapse would follow. What the paper does not provide is that full parallelization. The query phase is parallelizable in work, but compilation and counting are the expensive parts.

Given a DSOP of size K, rank reduces to sums of K counters and is in NC in n and K. select is obtained by binary search with $O(n)$ queries to rank; the search adds a factor n to depth, so the described algorithm is not necessarily NC, but its total work is $O(n^2K)$. If K is polynomial in the input size, the work is polynomial and the engine is in P; this is exactly Corollary 5.4. Parallelization does not remove the condition "given the certificate"; it only distributes the work.

If K is exponential, the work is exponential. By the relation $W = T \cdot P$, running it in polylogarithmic time requires $P = W/\mathrm{polylog}(N)$ processors, which is also exponential; that is not NC. Constructing the DSOP has work proportional to the number of pieces T of COVER-TRACE; if T is exponential, the work is exponential. The output itself has K cubes; if K is exponential, no algorithm can produce it with polynomial work, parallel or sequential. The strong separation between polynomial-size CNF and polynomial-size DSOP is no longer an open problem: Proposition 4.8 establishes it. What remains open is the separation for ADSOP, ZDD, and Fourier.

The comparator reduction is in NC: the comparator can be built with $O(n)$ gates and depth $O(\log n)$ using a parallel prefix. But the reduction calls the counter A; parallelizing it does not reduce the work of A. If A has work $2^{o(n)}\,\mathrm{poly}(n, m)$, the composition inherits that work; if it had polynomial work, it would already give $\#\mathsf{SAT} \in \mathsf{P}$ and $\mathsf{P} = \mathsf{NP}$. The reduction distributes the call, not the oracle.

The ZDD illustrates the same separation: the query has work $O(n)$ given the ZDD, but direct construction from the CNF is the open problem. A parallel construction with polynomial work would, by Proposition 7.7, be a polynomial sequential construction; then the engine would give $\#\mathsf{SAT} \in \mathsf{P}$ and the collapse would follow. The paper does not provide that construction.

Point queries are NC, but they compute a single bit. Computing all 2^n bits in parallel with 2^n processors has work 2^n; parallelization does not create aggregate information. Likewise, exhaustive search over the 2^n assignments with 2^n processors would solve SAT in time $O(nm)$, but with work $2^n \cdot nm$; that is not a polynomial algorithm in the P or NC model. The model counts total work, not only wall-clock time. Allowing exponential hardware does not prove $\mathsf{P} = \mathsf{NP}$; it changes the cost model.

In summary, parallelization does not eliminate the arguments of this section: it reinforces them. Given a polynomial-size certificate, the engine has polynomial-work queries; rank is a parallelizable sum and is in NC, while select retains a sequential search and adds a factor n to depth, although its work remains polynomial. The difficulty is in constructing the certificate or in counting models, and that difficulty is measured in total work, not depth. If the paper had given an NC algorithm for $\#\mathsf{SAT}$ for every CNF, sequential simulation would give a polynomial algorithm and then $\mathsf{P} = \mathsf{NP}$ would follow; but the paper does not give that algorithm.

7.13 What would be needed for a collapse

The engine does not imply $\mathsf{P} = \mathsf{NP}$ because none of its routes provides a uniform polynomial algorithm for $\#\mathsf{SAT}$. A collapse would require one of five things: a uniform polynomial DSOP compiler in the input size with polynomial output, a polynomial exact counter, a uniform polynomial ZDD compiler in the input size with polynomial output, a uniform polynomial ADSOP compiler, or a Walsh–Hadamard spectrum of polynomial size constructed in polynomial time. All imply $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$. The first is ruled out as a universal route by Proposition 4.8; four remain. The paper proves that queries are polynomial once one of those structures is available; it does not prove that the structure is available in polynomial time. That is the exact frontier: the engine is a query layer, not a collapse. As Section 7.12 shows, parallelization does not change this conclusion: an NC algorithm for the whole engine would imply $\mathsf{P} = \mathsf{NP}$, but the paper does not provide it.

The distinction is not a formality. If "querying a DSOP" were confused with "compiling a DSOP", the hypothesis of Theorem 7.1 would be obtained by definition and one would conclude $\mathsf{PH} = \mathsf{P}$. The error would be the same as confusing a certificate verifier with a certificate finder. The engine is a query and verification layer; finding the certificate is the open problem.

7.14 Program toward $\mathsf{P} = \mathsf{NP}$

The map above suggests a program with five fronts. It is not a wish list: each front has a falsifiable statement and, in several cases, an immediate consequence for collapse.

(P1) *Lower bounds for ADSOP.* Prove or refute Conjecture 7.5. A $2^{\Omega(n)}$ bound for a polynomial-size family would close the affine route; the random 3-CNF experiment is evidence, not proof.

(P2) *Affine backdoors.* Prove or refute Conjecture 7.6. This is the most computationally attackable front: one can search for backdoors in small instances and measure their growth.

(P3) *Direct ZDD and Fourier compilation.* Build those structures without going through the DSOP. A lower bound or a compiler for a rich family would decide the route.

(P4) *Proof complexity.* Compare K with resolution, *cutting planes*, and *polynomial calculus*; Proposition 4.8 is a first data point.

(P5) *A universal compiler or a polynomial counter.* This is the final goal; it must be uniform, deterministic, with polynomial output, and for every CNF.

The program can be summarized by two extremes. At one extreme, a universal polynomial compiler for the affine route (Conjecture 7.4) would give $\mathsf{P} = \mathsf{NP}$. At the other, an affine-incompressible family (Conjecture 7.5) would close the route. Between them lie intermediate situations: a superpolynomial but not exponential lower bound, universality only for structured families, or a different representation. The paper resolves neither extreme; it states them so that subsequent work knows which result would count as progress. The same applies to ZDD and Fourier.

The conjectures have different status. Conjecture 7.4 is the strongest in the paper and we have no direct evidence in its favor; we include it because it fixes the logical target. Conjecture 7.5 is more plausible in light of random 3-CNFs, but it is also open. The program does not consist in choosing one of the two: it consists in finding the first family or the first compiler that decides the question.

No result in this paper depends on Conjectures 7.4–7.6. They are included as a research program, separated from the theorems and propositions that are proved.

8 Hybrid algorithm

Theorem 8.1 (Correctness of the hybrid engine)**.** *Every branch of Algorithm 1 that terminates successfully returns the correct values of* bit, rank_0, rank_1, select_0, select_1, $\#\mathsf{SAT}$, *and uniform sampling for* $S(F)$.

Proof. The materialized branch uses the definition of the bitvector. The DSOP branch uses Theorem 5.3. The counting branch uses Theorem 6.2. The function bit is Proposition 3.1 in the non-materialized branches and the bitvector definition in the materialized one. Binary search and sampling are correct by monotonicity of rank_0 and the bijection $R \mapsto \mathrm{select}_0(R)$. □

Algorithm 1 Query engine for $S(F)$

Require: CNF F with n variables and m clauses; thresholds $N_0, K_{\max}$; exact counter A (optional)
Ensure: Operations $\mathrm{bit}, \mathrm{rank}_0, \mathrm{rank}_1, \mathrm{select}_0, \mathrm{select}_1, \#\mathsf{SAT}$ and sampling

if $n \le N_0$ **then**
  materialize $S(F)$ and build a succinct *rank/select* bitvector
else
  run COVERTRACE with cap $K_{\max}$
  if success and $K \le K_{\max}$ **then**
    build the DSOP engine of Theorem 5.3
  else if A is available **then**
    use Theorem 6.2 for rank and binary search for select
  else
    return UNAVAILABLE for aggregate queries
  end if
end if
function bit(k):
if bitvector materialized **then**
  return bit k of $S(F)$ $\{O(1)\}$
else
  return $1 - F(\mathrm{assign}(k))$ $\{O(m)\}$
end if

The cap $K_{\max}$ limits the output size of COVERTRACE; it does not guarantee that the search is polynomial. The algorithm selects representations; it does not bound the compilation cost. Correctness is conditional on the success of the branch, and Section 7 analyzes why that distinction prevents a collapse of classes.

Query	DSOP (K cubes)	Exact counter A	Materialized
bit(k)	$O(m)$	$O(m)$	$O(1)$
#SAT	$O(K)$	$T(n+O(n), m+O(n))$	$O(2^n/w)$
rank(k)	$O(nK)$	$T(n+O(n), m+O(n))$	$O(1)$
select(r)	$O(n^2K)$	$O(n\,T(n+O(n), m+O(n)))$	$O(1)$
sampling	$O(n^2K)$	$O(n\,T(n+O(n), m+O(n)))$	$O(1)$

Table 2: Work per query; DSOP and bitvector preprocessing are not included. w is the word size. The A column is conditional on the availability of the exact counter.

Observation 8.2 (Work and depth)**.** The bounds in Table 2 are work per query once the representation is built; they are not parallel depth bounds. In a PRAM model, bit has depth $O(\log m)$ with a reduction over clauses; #SAT and rank are sums and prefixes with depth $O(\log n+\log K)$; select retains the n-step binary search and, even parallelizing rank and computing prefix counts as parallel prefixes, its depth is $O(n(\log n+\log K))$ with the same work $O(n^2K)$. The counter branch inherits the depth D_A of A and the factor n of the search, so its depth is $O(n(D_A+\log n))$; the materialized branch pays $O(m2^n)$ construction work, although its queries are $O(1)$. Section 7.12 proves that an NC algorithm for the complete engine would imply P = NP; parallelizing the query phase of a given representation does not change the work bounds or the collapse condition.

The DSOP column counts arithmetic operations on $O(n)$-bit counters, except bit, which is measured in clause evaluations; in the bit model they are multiplied by $O(n/\log n)$. The materialized column assumes a succinct *rank/select* index. Table 2 summarizes the tradeoff. The materialized branch is only viable for small n; the DSOP branch is best when compilation is compact; the counting branch can answer for a k-CNF with exponential DSOP, at the cost of depending on an exact counter and paying one counting query per rank query.

Observation 8.3 (The bit in the compressed branches)**.** In the DSOP and counter branches, bit(k) is answered by evaluating F; one need not build the DSOP or call the counter for a point query. In the materialized branch, it suffices to read bit k of the bitvector. Thus a point query does not force payment of the DSOP compilation or a counter call; in the materialized branch, the $O(1)$ cost presupposes that the bitvector was already built for other queries.

8.1 What the cap $K_{\max}$ guarantees

The cap $K_{\max}$ bounds the output size of COVERTRACE, not the search time. If COVERTRACE succeeds and produces $K \le K_{\max}$, Theorem 5.3 gives queries polynomial in n and K; but the time to find that DSOP is not bounded by $K_{\max}$ and may be exponential. Something analogous happens with the other branches: the materialized branch has $O(1)$ queries but $O(m2^n)$ preprocessing, and the counter branch inherits the cost of A.

No polynomial bound is guaranteed: there is no theorem ensuring that the DSOP search or the counter call terminates in polynomial time for every CNF. Nor does the paper prove that the algorithm is exponential in the worst case; #P-hardness is a property of the exact problem, not of every branch of the hybrid. An absence of an upper bound does not imply a lower bound, and a lower bound for the problem does not translate into one for every instance.

Algorithm 1 is a *partial* algorithm: it may return UNAVAILABLE. It is not a uniform algorithm that solves #SAT for every CNF; that is why it contradicts neither #P nor H-ETH. If it were completed with an enumeration in the UNAVAILABLE branch, the worst case would be exponential

under H-ETH, but that completion is not part of the engine. Correctness is conditional on the success of the branch, as Theorem 8.1 already states.

8.2 Why the hybrid is not inevitably exponential

$\mathsf{P} \neq \mathsf{NP}$ does not imply that every SAT instance is hard; it implies that there is no uniform polynomial algorithm for all of them. Something analogous holds for #SAT: #P-hardness and H-ETH are worst-case statements. A family with a polynomial-size DSOP and a polynomial-time compiler, bounded treewidth, a small backdoor, or a polynomial-size ADSOP with a polynomial-time compiler can be solved in polynomial time; the engine exploits the representation when it is available. The same CNF may be easy for one representation and hard for another; the hybrid chooses.

There are therefore three levels that should not be confused:

(i) *Query given a representation.* With a DSOP of size K it is polynomial in n and K (Theorem 5.3); with an exact counter A it costs one call to A per rank query. It does not depend on $\mathsf{P} = \mathsf{NP}$.

(ii) *Construction of the representation or the counter.* It is specific to the instance and the family; it may be polynomial (wide clause, small DSOP, bounded treewidth, affine ADSOP) or exponential. No uniform polynomial compiler is known; one would give $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$.

(iii) *Exact problem in the worst case.* It is #P-hard; under H-ETH or #ETH there is no $2^{o(n)}$ algorithm for every CNF. This does not say that each instance is hard, nor that the concrete hybrid is exponential on every family.

The engine is a *portfolio*: it tries several representations and pays the cost of the attempt plus that of the branch that succeeds. If a branch is polynomial on a family and the attempt is also polynomial, the hybrid is polynomial on that family, even if the uniform worst case remains exponential.

Improvements that do not show up in the theoretical bound are real: better variable order, preprocessing, component decomposition, caching, succinct structures, parallelization, better SAT solvers, and better compilers can reduce work by a polynomial factor or change the practical regime without changing the worst-case bound. Theory says which branches are polynomial in their parameters and which barriers exist; it does not predict the time on a concrete instance. That is why the evaluation of the engine is experimental: it measures scaling, crossovers, and branch success rates, and does not infer a complexity bound from benchmarks.

An improvement may be theoretical for a subclass —polynomial K, bounded treewidth, small backdoor, ADSOP for parity— and experimental for the rest. The paper should present both separately: conditional theorems and barriers on one side; the portfolio and its evaluation on the other. The question "is it inevitably exponential?" has a graded answer: not for every instance; yes as a uniform worst case under standard conjectures if a total algorithm is required; and no uniform bound is known for the concrete hybrid; each family must be analyzed or measured.

8.3 Success conditions per branch

The useful question is not "is the engine polynomial for every CNF?" but "which families make a branch polynomial, and how much does it cost to decide that the instance belongs to that family?". The following proposition formalizes the sufficient condition.

Proposition 8.4 (Sufficient condition per family)**.** *Let $\mathcal{F}$ be a family of CNFs and R a representation with parameter $p_R(F)$. If there exist*

(i) a membership test for $\mathcal{F}$ in time $\mathrm{poly}(n, m)$*;*

(ii) a compiler that produces $R(F)$ in time $\mathrm{poly}(n, m, p_R(F))$*;*

(iii) a bound $p_R(F) \le \mathrm{poly}(n, m)$ *for every* $F \in \mathcal{F}$,

then the engine answers $\mathrm{bit}, \#\mathsf{SAT}, \mathrm{rank}, \mathrm{select}$*, and sampling in time* $\mathrm{poly}(n, m)$ *for every* $F \in \mathcal{F}$*. In the counter branch,* R *denotes the counter* A *and (ii) is interpreted as availability of* A *at cost* T*.*

Proof. The test chooses the branch; the compiler produces $R(F)$; queries are polynomial in n and $p_R(F)$ by Theorems 5.3 and 6.2 and by the definition of the materialized or stochastic branch. Composing the three bounds gives the result. If $\mathcal{F}$ is a promise class, test (i) is omitted; if the engine must decide whether the instance belongs to the family, (i) is part of the cost. □

Observation 8.5 (The family must be proper)**.** If $\mathcal{F}$ is all CNFs and (i)–(iii) hold for an exact representation or an exact counter, then $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$. Therefore the program of success conditions must produce proper families or parameters; covering all CNFs without a collapse is not possible with an exact representation and a polynomial test.

Table 3 summarizes the branches, their parameters, and the known sufficient conditions. The query column is work *given the witness*; it does not include the cost of finding it.

Branch	Witness / parameter	Query (work)	Sufficient condition	Bottleneck / what is missing
Materialized	$S(F)$	$O(1)$	$n \le N_0$	Construction $O(m2^n)$.
DSOP	K cubes	$O(nK)$, $O(n^2K)$	$K \le$ poly; bounded treewidth/pathwidth, few clauses, small backdoor	COVERTRACE and coverage verification.
Counter A	cost T	T, nT	A efficient on the family (affine, treewidth, backdoor)	A polynomial counter for every CNF would collapse.
ZDD	size Z	$O(n)$	$Z \le$ poly; bounded pathwidth/treewidth	Build the ZDD and choose the order.
Fourier	sparsity s	$O(sn)$, $O(sn^2)$	$s \le$ poly; parity, junta, sparse spectrum	Build or detect the spectrum.
ADSOP	A affine pieces	$O(A)$, $O(n^4A)$	$A \le$ poly; parity, affine subspaces, affine backdoor	Compiler and lower bound.
Stochastic	ε, δ	approximate	non-negligible density; hashing	Exactness; complement cancellation.

Table 3: Branches, parameters, and sufficient conditions. The query is work given the witness; the cost of finding the witness or deciding membership is not included.

The table separates two problems that are often confused: (a) given the representation, the query is polynomial; (b) deciding that the instance is in the family and building the representation may be hard. A success condition is not an algorithm until the test and the compiler are polynomial. Useful research consists in enlarging the families and lowering the test cost, not in postulating that every CNF is in one. Parity is the witness case: DSOP is exponential, but ADSOP, the spectrum, and an affine counter are polynomial; a wide clause is a DSOP of size one; a bounded-treewidth CNF falls into the counter or the ZDD; a random 3-CNF near the threshold may fall outside all known conditions, although for small n the size of $U(F)$ may make the DSOP of U small.

Regime pilot. The script `branch_regime_map.py` computes the parameters in Table 4 for $n = 10$. It does not measure query times; its purpose is to check that the branches are not an empty classification: parity has $K = 512$ and ADSOP 1; the wide clause has $K = 1$; the chain has estimated treewidth 2; the 3-CNF with $r = 2$ has no cheap exact branch, while the instances

with $r = 3$ and $r = 4.2$ have few solutions and their forbidden region is almost the whole cube, which reduces K.

Family	m	vol(U)	#SAT	K	A	s	tw	Viable branches
wide	1	32	992	1	1	32	4	DSOP, ADSOP, Fourier, A
parity	512	512	512	512	1	2	9	ADSOP, Fourier, affine A
cube+parity	768	768	256	512	2	4	9	ADSOP, Fourier, affine A
3-CNF $r = 2$	20	943	81	285	281	1024	6	stochastic
3-CNF $r = 3$	30	1019	5	38	38	1024	7	DSOP, ADSOP, stochastic
3-CNF $r = 4.2$	42	1021	3	25	25	1024	7	DSOP, ADSOP, stochastic
chain	8	520	504	306	144	586	2	treewidth A, stochastic
components	3	338	686	162	40	512	2	ADSOP, component A, stochastic

Table 4: Regime pilot at $n = 10$. K and A are the outputs of `cover` and `affine_cover`; tw is an estimate by minimum elimination; the materialized branch is omitted because $n \leq N_0$. No times or scaling are measured.

Tensor networks and rank. An additional, still theoretical branch is the contraction of a tensor network with low-rank cuts. The parameter is the cut rank or contraction width; the query is a contraction; the sufficient condition is that this rank be polynomial; the bottleneck is finding the decomposition. The Anthropic manuscript [6] uses a hierarchy of cuts, an eight-square identity, and a bit-balancing argument to obtain a $2^{o(n)}$ algorithm for #k-SAT. It is not a practical branch: there is no implementation, the constant is not explicit, and the manuscript itself rules out improvements until $n > 10^{10\,000}$ in the case $m = 5n$. It does not replace a DSOP or ADSOP compiler either: its output is a subexponential counter, not a representation that answers rank or select.

9 Optimization via ZDD and Fourier

The structure of Theorem 5.3 answers rank in $O(nK)$ because it sums the prefixes of K cubes. A natural optimization is to compile the disjoint family into an ordered *Zero-suppressed Decision Diagram* (ZDD) [11, 10], with variable order $x_1, \ldots, x_n$, in which each node stores the number of assignments of its subgraph. We agree that the ZDD represents the family $U(F)$ of assignments that falsify F; the number of sets in the ZDD is $|U(F)|$ and $\#\mathsf{SAT}(F) = 2^n - |U(F)|$. In a ZDD, bit, rank, select, and sampling are answered by traversing at most n levels, once the structure is built.

If the ZDD is built from the DSOP of K cubes, its size is $O(nK)$: the trie of the K patterns has at most nK nodes and reduction can only decrease it. The advantage is that queries drop from $O(nK)$ to $O(n)$. The interesting question is whether a ZDD much smaller than nK can be built directly from the CNF, without going through the DSOP. Parity is the witness case: its minimum DSOP has 2^{n-1} cubes, but its ZDD has $O(n)$ nodes, because the family is described by a linear equation. Building that ZDD directly from the CNF is an open problem (Section 13).

The ZDD is therefore an engineering component inside the hybrid engine: if the compressed structure is small, queries drop to $O(n)$; otherwise, the DSOP engine is kept or the counting oracle is used.

9.1 Fourier representation

The truth table $S(F)$ also admits a representation by its Walsh–Hadamard spectrum [8]. Let $f(x) = S(F)[x]$. For $S \subseteq [n]$, the Fourier coefficient is

$$\hat{f}(S) = \sum_{x \in \Omega_n} f(x)\,(-1)^{\sum_{i \in S} x_i}.$$

The inverse transform is

$$f(x) = \frac{1}{2^n} \sum_{S \subseteq [n]} \hat{f}(S)\,(-1)^{\sum_{i \in S} x_i}.$$

The empty coefficient is $\hat{f}(\varnothing) = |U(F)|$, so $\#\mathsf{SAT}(F) = 2^n - \hat{f}(\varnothing)$. Let $s = |\{S \subseteq [n] : \hat{f}(S) \neq 0\}|$ be the *density* of the spectrum.

If the s nonzero coefficients are available, $\mathrm{rank}_1(k)$ is computed as

$$\mathrm{rank}_1(k) = \frac{1}{2^n} \sum_{S : \hat{f}(S) \neq 0} \hat{f}(S) \sum_{\mathrm{pos}(x) < k} (-1)^{\sum_{i \in S} x_i}.$$

The inner prefix sum is computed in $O(n)$ with the two-state automaton of Lemma 5.1, adapted to characters. Therefore rank costs $O(sn)$, select costs $O(sn^2)$ by binary search, and sampling costs $O(sn^2)$. The Fourier engine is exact: the coefficients are integers and the division by 2^n is accumulated at the end.

The representation is useful when s is small. Parity has $s = 2$: $S(F_n)$ is the indicator of odd parity and its spectrum is concentrated on $\varnothing$ and $[n]$, with coefficients 2^{n-1} and -2^{n-1}. In contrast, a wide clause of width w has $s = 2^w$: its falsifying cube fixes w coordinates and the spectrum is distributed over the subsets of those coordinates. A random Boolean function has a dense spectrum: most of the 2^n coefficients are nonzero. There is therefore no universal compression guarantee.

The algorithm of [7] accelerates the DFT of length N by a polylogarithmic factor, but for $N = 2^n$ it remains exponential in n; it does not replace a sparse-spectrum compiler. The relevant question for the engine is whether the spectrum of $S(F)$ can be built with s polynomial in polynomial time; the empty coefficient is already $|U(F)|$, so building the spectrum for every CNF is at least as hard as model counting.

The Fourier representation is therefore a third compression component, complementary to DSOP and ZDD: it wins for parity and functions with sparse spectrum, and loses for wide clauses or random functions. Section 7 includes it as one more route to collapse.

10 Stochastic oracles and affine covers

Sections 5–9 assume that the DSOP, the ZDD, or the spectrum is already built. This section addresses the construction phase from two additional angles. The first is stochastic: the exact answer is replaced by a sample or by an estimator with controlled error. The second is algebraic: cubes are generalized to affine subspaces over $\mathbb{F}_2$, so that parity is no longer the negative witness of DSOP. Neither family removes the #P barrier; both shift the cost and change the type of guarantee.

10.1 Oracle types and separations

For a partial assignment $v \in \{0, 1, \bullet\}^n$, let $Q(v)$ be the subcube of assignments extending v, and let $F|_v$ be the CNF obtained by fixing the variables of v; thus $U(F|_v) = Q(v) \cap U(F)$. A query interface over the cube may offer five different operations:

- $\mathsf{bit}(x)$: the value $f(x)$ at a complete assignment; it is local and costs $O(m)$;
- $\mathsf{has1}(v)$: whether $Q(v) \cap U(F) \neq \emptyset$; this is a satisfiability query;
- $\mathsf{sample}(v)$: a uniform sample from $Q(v) \cap U(F)$;
- $\mathsf{est}(v, \varepsilon, \delta)$: a multiplicative estimate of $\mathrm{vol}(Q(v) \cap U(F))$;
- $\mathsf{count}(v)$: the exact volume $\mathrm{vol}(Q(v) \cap U(F))$.

Table 5 summarizes what can be obtained from each. The difference between $\mathsf{has1}$ and count is the difference between deciding and counting; the difference between sample and count is the difference between a distribution and a normalization.

Interface	Answer	Witness	Exact #SAT	Approximate #SAT	rank / select
$\mathrm{bit}(x)$	$f(x)$	not global	2^n queries	only if density is high	no
$\mathsf{has1}(v)$	$Q(v) \cap U(F) \neq \emptyset$	$O(n)$	not known	$\mathsf{BPP}^{\mathsf{NP}}$	not exact
$\mathsf{sample}(v)$	uniform in $Q(v) \cap U(F)$	$O(1)$	no	implies approximate counting	approximate
$\mathsf{est}(v, \varepsilon, \delta)$	$(1 \pm \varepsilon)$ with confidence	no	no	yes	approximate
$\mathsf{count}(v)$	$\mathrm{vol}(Q(v) \cap U(F))$	$O(n)$	one query	yes	yes
Sparse WHT	coefficients	yes	if s is poly	yes	yes
ADSOP	disjoint union of affines	yes	$O(K)$	yes	yes

Table 5: Interface types and the guarantee each offers. The exact #SAT column refers to a uniform algorithm for every CNF; the approximate one is with relative error.

Proposition 10.1 (Decision does not give exact counting)**.** *If there were a polynomial deterministic algorithm with a* $\mathsf{has1}$ *oracle that computed* $\#\mathsf{SAT}(F)$ *for every CNF, then* $\#\mathsf{P} \subseteq \mathsf{FP}^{\mathsf{NP}}$ *and, by Toda's theorem,* $\mathsf{PH} = \Delta_2^{\mathsf{P}} = \mathsf{P}^{\mathsf{NP}}$.

Proof. The algorithm would give $\#\mathsf{SAT} \in \mathsf{FP}^{\mathsf{NP}}$. By completeness of #SAT for #P, it follows that $\#\mathsf{P} \subseteq \mathsf{FP}^{\mathsf{NP}}$ and therefore every polynomial machine with a #P oracle is simulated with an NP oracle, that is, $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}^{\mathsf{NP}}$. By Toda, $\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}^{\mathsf{NP}} = \Delta_2^{\mathsf{P}} \subseteq \mathsf{PH}$, whence $\mathsf{PH} = \Delta_2^{\mathsf{P}}$. □

Observation 10.2 (Approximating is not counting)**.** Approximate counting with relative error and confidence is possible with a polynomial probabilistic algorithm that queries an NP oracle (Stockmeyer [21]); no polynomial exact algorithm is known. It is not known that $\mathsf{P} = \mathsf{NP}$ implies $\#\mathsf{SAT} \in \mathsf{FP}$. A polynomial exact compiler would imply $\#\mathsf{SAT} \in \mathsf{FP}$, but no implication in the opposite direction is known.

10.2 Sampling, hashing, and Karp–Luby cancellation

Direct sampling. Let $p = \#\mathsf{SAT}(F)/2^n$. If we sample $x \sim \Omega_n$ and evaluate $F(x)$, the estimator

$$\hat{p} = \frac{1}{T} \sum_{j=1}^{T} F(x_j), \qquad \widehat{\#\mathsf{SAT}} = 2^n \hat{p}$$

is unbiased. Hoeffding gives

$$\Pr\big(|\hat{p} - p| \geq \varepsilon\big) \leq 2e^{-2T\varepsilon^2}.$$

The additive error is cheap: $T = O(\log(1/\delta)/\varepsilon^2)$. The relative error is the problem:

$$\frac{\Delta\#\mathsf{SAT}}{\#\mathsf{SAT}} \approx \frac{1}{\sqrt{Tp}}.$$

If $p = 2^{-n}$ (a single solution), $T \approx 2^n/\varepsilon^2$ samples are needed. When the density is exponentially small, uniform sampling requires an exponential number of samples and Hoeffding's bound ceases to be useful in practice.

XOR hashing. Add t random parity constraints

$$H_t : \quad \bigoplus_{i \in S_j} x_i = b_j, \qquad j = 1, \dots, t,$$

with $S_j \subseteq [n]$ a nonempty random subset and b_j uniform in $\{0, 1\}$. For a fixed solution, each of the t random parities is satisfied with probability $1/2$; hence the solution survives with probability 2^{-t}. The rank of the constraints does not change that probability: it only determines how many solutions the hashed system has. Then

$$\mathbb{E}\big[\mathrm{count}(F \wedge H_t)\big] = \frac{\#\mathsf{SAT}}{2^t}, \qquad \widehat{\#\mathsf{SAT}}_t = \mathrm{count}(F \wedge H_t)\, 2^t$$

is unbiased. The estimator we implement is exactly $\widehat{\#\mathsf{SAT}}_t$, with count computed by an exact solver on the hashed CNF.

Unbiasedness is not enough for a relative-error guarantee. The ApproxMC analysis [23] obtains $(1 \pm \varepsilon)$ with probability $1 - \delta$ under a t-wise independent hash family and an adaptive choice of t that keeps the expected count in a constant range; the number of repetitions and the constant depend on ε and δ. Our prototype does not implement the full t-wise independent family or the certified adaptive choice: it uses random subsets, a cap of 64 models, seven repetitions, and a heuristic for t. Therefore the formal guarantee is that of the idealized algorithm, not of the prototype. Table 6 reports the observed error, not an error bound; an unbiased estimator can have large relative error, and the median of a few repetitions does not replace a concentration proof.

Table 6 summarizes the experiment with 120 000 direct samples and an ApproxMC prototype with a cap of 64 models and seven repetitions. The cases with few solutions illustrate the difference: direct sampling does not detect the solution with useful probability, while hashing reduces the space and locates it.

Case	#SAT	Direct	Rel. error	Hashing t	Hashing	Rel. error
unique solution $n = 20$	1	0	100%	0	1	0%
disjoint $n = 20$	470 596	472 768	0.46%	13	471 040	0.09%
disjoint $n = 24$	5 764 801	5 747 874	0.29%	17	6 291 456	9.14%
3-CNF $n = 24$, $m = 72$	1 838	1 957	6.49%	6	1 984	7.94%
3-CNF $n = 26$, $m = 78$	3 081	2 237	27.39%	6	3 328	8.02%
UNSAT $n = 18$, $m = 75$	0	0	0%	0	0	0%

Table 6: Direct sampling versus XOR hashing. Hashing is approximate and the table reports the observed error of a prototype, not an FPRAS guarantee. The unique-solution case with $t = 0$ is exact because the number of models is already small and is enumerated.

Karp–Luby and complement cancellation. There is a route that seems to solve the problem and does not. The forbidden region

$$U(F) = \bigcup_{C \in F} Q(p(C))$$

is a DNF with m terms. Karp–Luby [20] estimates the size of a union of sets:

$$N = \sum_{C \in F} \mathrm{vol}(p(C)), \qquad \widehat{|U|} = \frac{1}{T} \sum_{j=1}^{T} \frac{N}{c(x_j)},$$

where a cube C is chosen with probability $\mathrm{vol}(p(C))/N$, x_j is sampled uniformly in $Q(p(C))$, and $c(x_j)$ is the number of cubes containing x_j. The estimator is unbiased and gives an FPRAS for $|U|$ in time $\mathrm{poly}(n, m, 1/\varepsilon, \log(1/\delta))$.

The problem is the subtraction:

$$\#\mathsf{SAT} = 2^n - |U|.$$

If $|U|$ is estimated with relative error ε, the absolute error is $\varepsilon|U|$, and the relative error of $\#\mathsf{SAT}$ is

$$\frac{\varepsilon|U|}{\#\mathsf{SAT}}.$$

When $\#\mathsf{SAT}$ is a tiny fraction of 2^n, that factor is enormous; the estimate can even be negative. Table 7 shows this.

Case	$\#\mathsf{SAT}$	Estimated $\lvert U\rvert$	Estimated $\#\mathsf{SAT}$	Rel. error
unique solution $n = 20$	1	1 047 980.6	595.4	59 441%
disjoint $n = 24$	5 764 801	11 031 257.6	5 745 958.4	0.33%
3-CNF $n = 24$, $m = 72$	1 838	16 797 232.4	−20 016.4	1 189%
3-CNF $n = 20$, $m = 40$	5 198	1 046 867.7	1 708.3	67.14%

Table 7: Karp–Luby with 50 000 samples estimates $|U|$ well, but the subtraction $2^n - |U|$ destroys the relative accuracy of $\#\mathsf{SAT}$ when the solution density is tiny.

Observation 10.3 (Counting the forbidden DNF is not counting SAT)**.** The Karp–Luby FPRAS is a stochastic oracle for $U(F)$, not for F. Complement cancellation is the counterpart of direct sampling: in one case one estimates the solution region, which may be tiny; in the other, the complementary region, which may be almost the whole cube. The estimator does not impose the bound $|U| \leq 2^n$; that is why the subtraction can produce a negative value, as in Table 7. This does not contradict the FPRAS: the relative error of $|U|$ is small, but that of $\#\mathsf{SAT}$ is not. The absurdity is the lesson, not an implementation bug: an estimate with small relative error on the complement can be useless for the original set when it is tiny. An FPRAS for $\#\mathsf{SAT}$ would require controlling the absolute error of $|U|$ to precision $O(\#\mathsf{SAT})$, which is exponential in the rare regime.

Partitions, belief, and statistical physics. A random partition of subcubes with $\mathsf{has1}$ and $\mathsf{has0}$ can prune when the cube is homogeneous; if the tree closes, the count is exact; if it stops at mixed leaves, one obtains a confidence interval. The number of distinct subcubes is related to the number of cofactors and to the size of the OBDD for that order. Monte Carlo methods over assignments (Metropolis, Gibbs, Wang–Landau) and belief propagation or *survey propagation* [24] estimate marginals or state densities; they are exact on trees and heuristics on graphs with cycles. They are not suitable for exact $\#\mathsf{SAT}$; for large, dense instances they are a practical estimation alternative.

Random affine subspaces. XOR constraints are intersections with affine subspaces. Instead of sampling bits, one samples an affine subspace of dimension $n - r$; this is exactly what hashing does. The difference from bit sampling is that the subspace can be exponentially large and still manageable by linear algebra. It is the bridge between the stochastic part and the affine cover of Section 10.3.

10.3 Affine cover: ADSOP

DSOP covers $U(F)$ with cubes, and a cube is a particular case of an affine subspace over $\mathbb{F}_2$. The natural generalization is a *disjoint sum of affine subspaces* (ADSOP):

$$U(F) = \bigsqcup_{j=1}^{K} A_j, \qquad A_j = p_j + \langle b_{j,1}, \ldots, b_{j,d_j} \rangle_{\mathbb{F}_2},$$

where $b_{j,1}, \dots, b_{j,d_j}$ are linearly independent. Each piece has exactly 2^{d_j} points and $d_j = \dim A_j$. DSOP is the case in which the $b_{j,\ell}$ are unit vectors and p_j fixes the other coordinates.

Theorem 10.4 (Exact queries on an ADSOP)**.** *Let $U(F) = \bigsqcup_{j=1}^{K} A_j$ be an ADSOP with $\dim A_j = d_j$ and $A_j = p_j + \langle b_{j,1}, \dots, b_{j,d_j} \rangle$. For a pattern v, let $M_{j,v}$ be the linear system in variables $c_1, \dots, c_{d_j}$ imposing*

$$\sum_{\ell=1}^{d_j} c_\ell \, (b_{j,\ell})_i = v_i - (p_j)_i \qquad \text{for each } i \in \operatorname{supp}(v).$$

Then:

(i) $|U(F)| = \sum_{j=1}^{K} 2^{d_j}$ *and* $\#\mathsf{SAT}(F) = 2^n - |U(F)|$*;*

(ii) a witness of $U(F)$ is obtained by solving a linear system in a nonempty piece;

(iii) for every pattern v,

$$|Q(v) \cap A_j| = \begin{cases} 2^{d_j - \operatorname{rk}(M_{j,v})}, & \text{if } M_{j,v} \text{ is consistent,} \\ 0, & \text{if } M_{j,v} \text{ is inconsistent,} \end{cases}$$

and $\operatorname{vol}(Q(v) \cap U(F)) = \sum_{j=1}^{K} |Q(v) \cap A_j|$*;*

(iv) rank *and* select *are computed by prefix counting and binary search in time polynomial in n and K.*

Proof. (i) is additivity of volumes for a disjoint union. (ii) is linear algebra: a point of A_j is p_j plus a combination of the $b_{j,\ell}$. For (iii), fixing the coordinates of v imposes one linear equation for each fixed variable on the coefficients of the combination. If the system is inconsistent, the solution set is empty and the contribution is 0; it is not replaced by 2^{d_j}. If it is consistent, the solution set is an affine variety of dimension $d_j - r$, where $r = \operatorname{rk}(M_{j,v})$, and has $2^{d_j - r}$ points. The sum over pieces is exact because they are disjoint. For (iv), the prefix automaton of Lemma 5.1 generates $O(n)$ partial patterns v, one per level and branch of the automaton; for each one apply (iii), and inconsistent branches contribute zero. rank sums those counts and select descends through the prefix bits using the same counts. With naive Gaussian elimination, one prefix count costs $O(n^3)$, so rank costs $O(n^4 K)$; select by binary search costs $O(n^5 K)$ and by prefix descent $O(n^4 K)$. Bitsets reduce the polynomial factor. □

Observation 10.5 (The inconsistent case is not cosmetic)**.** The correction in (iii) changes the value, not only the presentation. If A_j is the subspace $x_1 = 0$ in $n = 2$ and v fixes $x_1 = 1$, then $Q(v) \cap A_j = \varnothing$; the formula that assigns rank 0 to an inconsistent system would return $2^{d_j} = 2$, the volume of the piece, when the correct value is 0. An earlier version of this manuscript made exactly that mistake; it was detected by comparing prefix counts with full enumeration. That is why the experimental protocol of Section 11 includes empty intersections, zero-dimensional intersections, and consistent full-rank systems, and compares every count with the full table for $n \le 10$.

Algorithm 2 PrefixAffine(A_j, k)

Require: affine piece $A_j = p_j + \langle b_{j,1}, \ldots, b_{j,d_j} \rangle$; integer k with n bits
Ensure: number of points of A_j with $\mathrm{pos}(x) < k$

```
c ← 0; v ← empty pattern; tight ← true
for i = 1 to n do
  if tight then
    for b ∈ {0,1} with b < k_i do
      v' ← v ∪ {x_i = b}
      solve M_{j,v'}; if consistent, c ← c + 2^{d_j − rk(M_{j,v'})} {otherwise add 0}
    end for
    v' ← v ∪ {x_i = k_i}; if M_{j,v'} is consistent, v ← v'; otherwise break
  end if
end for
return c
```

Algorithm 2 makes point (iii) explicit: compatibility is checked before adding, and an inconsistent branch contributes 0. The total count is the sum over the K pieces; $\mathrm{rank}_0(k) = k - \sum_j \textsc{PrefixAffine}(A_j, k)$ and select_0 descends through the bits of k using those same counts. The correctness of the algorithm is that of Theorem 10.4; its cost is $O(n)$ linear systems per piece, each of cost $O(n^3)$ with Gaussian elimination, yielding the bounds $O(n^4 K)$ for rank and $O(n^4 K)$ for select by descent.

Why it breaks the parity bound. Odd parity

$$O_n = \{x \in \Omega_n : x_1 \oplus \cdots \oplus x_n = 1\}$$

is a single affine subspace of dimension $n-1$: ADSOP $K = 1$. The minimum DSOP needs 2^{n-1} cubes. A wide clause is also a cube, hence one piece. The union

$$U = \{x_0 = 0\} \cup \{x_0 = 1,\ \mathrm{parity}(x_1, \ldots, x_{n-1}) = 1\}$$

is an ADSOP of 2 pieces, while the minimum DSOP needs $1 + 2^{n-2}$ cubes.

Where it does not help. For a generic set, the affine hull is the whole space and ADSOP degrades to DSOP or to a worse partition. Table 8 summarizes the experiment at $n = 12$. The DSOP column is the output of the `cover` heuristic, not always the minimum: for cube+parity the minimum is $1 + 2^{n-2} = 1025$, while `cover` returns 2048.

Family	vol(U)	DSOP (`cover`)	ADSOP
parity	2048	2048	1
wide clause	64	1	1
cube + parity	3072	2048	2
random affine subspace (dim. 6)	64	64	1
random 3-CNF ($m = 2n$)	3969	627	627

Table 8: Heuristic DSOP versus ADSOP at $n = 12$. ADSOP compresses parity and subspaces; it does not compress a generic random CNF.

ADSOP and Fourier. The following examples show that neither representation dominates the other on every family. Parity has ADSOP 1 and spectrum $s = 2$. A wide clause of width w has ADSOP 1 and spectrum $s = 2^w$. A random affine subspace of dimension $n/2$ has ADSOP 1 and spectrum $s = 2^{n/2}$. For a random function no compression is expected in either representation, and the random 3-CNF experiment shows no ADSOP gain. Compactness therefore depends on the algebraic structure of the forbidden set.

10.4 Direct ADSOP compiler

The pending question is how to build the ADSOP from the CNF without enumerating $U(F)$. The prototype `sat_adsop_compiler.py` uses a SAT solver as an oracle and linear algebra over $\mathbb{F}_2$. The central piece is the affine hull of a set of points of $U(F)$.

Finding points of $U(F)$. For a partial assignment v, $U(F)$ is encoded as a DNF with selectors. For each clause C not satisfied by v, create a selector variable s_C and, for each unfixed literal $\ell \in C$, the clause $(\neg s_C \vee \neg \ell)$. The large clause $\bigvee_C s_C$ requires at least one clause to be falsified. A model of the system is a point of $U(F)$ extending v. Adding a linear equation $a \cdot x = b$ restricts the search to an affine subspace.

Lemma 10.6 (Encoding of $U(F|_v)$)**.** *Let $\mathcal{S}(F, v)$ be the CNF formed by the units of v, the clauses $(\neg s_C \vee \neg \ell)$ for each clause C not satisfied by v and each unfixed literal $\ell \in C$, and the clause $\bigvee_C s_C$. If there are no selectors, the large clause is empty and the system is unsatisfiable. Then $\mathcal{S}(F, v)$ is satisfiable if and only if $U(F|_v) \neq \emptyset$. Moreover, the projection of every model onto the original variables belongs to $U(F|_v)$, and every point of $U(F|_v)$ extends to a model.*

Proof. If $x \in U(F|_v)$, some clause C is falsified by x. Take $s_C = 1$ and $s_{C'} = 0$ for $C' \neq C$. The units of v are satisfied because x extends v; the clauses $(\neg s_C \vee \neg \ell)$ are satisfied because x falsifies the unfixed literals of C; the clauses with $s_{C'} = 0$ are satisfied by $\neg s_{C'}$; and the large clause is satisfied by s_C. Conversely, if there is a model, the large clause forces $s_C = 1$ for some C. The clauses $(\neg s_C \vee \neg \ell)$ then force all unfixed literals of C to be false; the fixed literals are false because C is not satisfied by v. Therefore C is falsified and the projection of the model belongs to $U(F|_v)$. The last statement follows from the first construction. □

With this lemma, FindU(F, v) is implemented by solving $\mathcal{S}(F, v)$ and projecting a model onto the original variables; if the system is unsatisfiable, it returns $\perp$.

Deciding whether $U(F)$ is affine. Let H be the affine hull of the points found. If $F|_v \wedge [x \in H]$ is satisfiable, H contains a point satisfying F and therefore $U(F|_v)$ is not affine. If it is not satisfiable, $H \subseteq U(F|_v)$. The equations of H are a basis of the orthogonal complement of its direction space; they are computed by Gaussian elimination. To check the other inclusion, traverse that basis: for each equation $a \cdot x = b$, search for a point of $U(F|_v)$ with $a \cdot x = 1 - b$. If none exists, every point of $U(F|_v)$ satisfies all equations and $U(F|_v) \subseteq H$; together with $H \subseteq U(F|_v)$, this gives $U(F|_v) = H$. If one exists, add the point to the hull and repeat. Each iteration increases the affine dimension by at least one, so the process terminates in at most $n+1$ iterations.

Algorithm 3 AffineClosure(F, v)

Require: CNF F, partial assignment v

Ensure: EMPTY, AFFINE with H, or NOT-AFFINE with a counterexample (or a termination error, an unreachable case)

```
p ← FindU(F, v)
if p = ⊥ then
  return EMPTY
end if
P ← {p}
for iter = 1, ..., n + 1 do
  H ← AffineHull(P)
  c ← model of F|_v ∧ [x ∈ H]
  if c ≠ ⊥ then
    return NOT-AFFINE with c and P
  end if
  q ← ⊥
  for each equation a · x = b of H do
    q ← FindU(F, v ∧ [a · x = 1 − b])
    if q ≠ ⊥ then
      break
    end if
  end for
  if q = ⊥ then
    return AFFINE with H and P
  end if
  P ← P ∪ {q}
end for
return NOT-AFFINE with a termination error
```

Lemma 10.7 (Strict growth of the hull)**.** *In an iteration of AffineClosure, let $H =$ AffineHull(P) and let $q \in U(F|_v)$ be a point that violates some equation of H. Then*

$$\dim \mathit{AffineHull}(P \cup \{q\}) > \dim H.$$

Proof. Since $q \notin H$, the affine hull $H' =$ AffineHull$(P \cup \{q\})$ strictly contains H. If $\dim H' = \dim H$, then H' would be an affine variety of the same dimension as H containing H; two affine varieties of the same dimension, one contained in the other, are equal, a contradiction. Therefore $\dim H' \geq \dim H + 1$. □

Lemma 10.8 (Correctness of AffineClosure)**.** *Under a partial assignment v, AffineClosure returns EMPTY if and only if $U(F|_v) = \emptyset$; it returns AFFINE with H if and only if $U(F|_v) = H$ is an affine subspace; and it returns NOT-AFFINE with a point satisfying H if $U(F|_v)$ is not affine.*

Proof. By Lemma 10.6, if $p = \bot$ the DNF encoding of $U(F|_v)$ is unsatisfiable and $U(F|_v) = \emptyset$. Otherwise, H is the affine hull of points of $U(F|_v)$, so H is affine and contains P. If $F|_v \wedge [x \in H]$ is satisfiable, H contains a point outside $U(F|_v)$; since every affine variety containing P contains H, $U(F|_v)$ cannot be affine. If it is unsatisfiable, $H \subseteq U(F|_v)$. If in addition no point of $U(F|_v)$ violates an equation of H, then $U(F|_v) \subseteq H$ and $U(F|_v) = H$. If some point violates an equation, it lies outside H and is added to P; by Lemma 10.7 the dimension of the hull increases by at least one in each iteration, so the loop terminates in at most $n + 1$ iterations. The error branch is not reached: the affine dimension is at most n. □

Algorithm 4 CompileADSOP(F, v)

Require: CNF F, partial assignment v (initially empty)
Ensure: a disjoint family of affine subspaces covering $U(F|_v)$

```
if F|_v is UNSAT then
    return {Q(v)} {the whole cube belongs to U}
end if
(status, H) ← AffineClosure(F, v)
if status = EMPTY then
    return ∅
end if
if status = AFFINE then
    return {H}
end if
choose a free variable i
return CompileADSOP(F, v ∪ {x_i = 0}) ∪ CompileADSOP(F, v ∪ {x_i = 1})
```

The choice of the free variable does not affect correctness. In the prototype, a variable that separates the points of U found during the hull is chosen to reduce the number of nodes; this heuristic is not part of the correctness theorem.

Theorem 10.9 (Correctness of the ADSOP compiler)**.** *The pieces returned by CompileADSOP are affine subspaces, are contained in $U(F|_v)$, are pairwise disjoint, and cover $U(F|_v)$.*

Proof. Induct on the recursion depth. If $F|_v$ is UNSAT, the whole cube $Q(v)$ is in $U(F|_v)$ and is affine. If AffineClosure returns EMPTY, Lemma 10.8 gives $U(F|_v) = \emptyset$ and the branch returns the empty set. If it returns AFFINE, the same lemma gives $H = U(F|_v)$, and $H \subseteq Q(v)$ because every point of H extends v. If it returns NOT-AFFINE, the two branches fix the free variable $x_i = 0$ and $x_i = 1$; their cubes are disjoint and cover $Q(v)$, and by induction each branch covers exactly its part of $U(F|_v)$. The union is disjoint and covers $U(F|_v)$. □

Proposition 10.10 (Cost of the prototype)**.** *Each call to AffineClosure makes $O(n^2)$ SAT queries and terminates in $O(n)$ iterations. The tree of CompileADSOP can have up to $2^{n+1}-1$ nodes, so the total number of SAT queries is $O(n^2\,2^n)$. The prototype has no polynomial bound in n and m.*

Proof. In each iteration one query $F|_v \wedge [x \in H]$ is made and, at most, one query for each of the n equations of H; the number of iterations is bounded by $n+1$ because, by Lemma 10.7, the dimension of H grows strictly. The tree fixes one variable per level and can reach 2^n leaves; multiplying by the $O(n^2)$ queries per node gives the bound $O(n^2\,2^n)$. Each SAT query is an NP instance; the prototype does not eliminate that hardness, it only avoids enumerating $U(F)$. □

The constraints $x \in H$ are encoded as XOR equations. The prototype translates them to CNF with auxiliary variables; alternatively, a solver with native XOR accepts them directly. The encoding preserves satisfiability, which is all the compiler needs; it does not preserve the number of models, but the compiler does not count models in those queries.

Table 9 compares the prototype with the brute affine cover and the heuristic DSOP on instances with $n \le 10$. All covers were validated against the full table: the pieces are disjoint and cover exactly $U(F)$.

Family	n	m	vol(U)	Direct ADSOP	Brute ADSOP	DSOP `cover`	SAT	nodes
parity	6	32	32	1	1	32	14	1
parity	8	128	128	1	1	128	18	1
parity	10	512	512	1	1	512	22	1
wide clause	10	1	32	1	1	1	18	1
cube+parity	8	192	192	2	2	128	27	3
cube+parity	10	768	768	2	2	512	31	3
3-CNF $m = 2n$	8	16	225	45	88	92	629	89
3-CNF $m = 2n$	10	20	943	134	281	285	1804	267

Table 9: Direct ADSOP compiler. The brute ADSOP column uses greedy partitioning with the full list of points; the DSOP `cover` column is the cube heuristic. The prototype does not use `brute_force` to build the pieces; final validation does compare with the full table. The comparison is of cover quality, not cost: the direct compiler uses a SAT oracle and may be exponential.

Observation 10.11 (The prototype produces fewer pieces than greedy partitioning)**.** On random 3-CNF the direct compiler produces fewer pieces than the brute affine cover (45 versus 88 at $n = 8$, 134 versus 281 at $n = 10$), because the hull with SAT detects a large affine piece before splitting. It is not minimal; it is a tighter upper bound. The parity and cube+parity CNFs are full expansions, with one clause per assignment of $U(F)$ ($m = 2^{n-1}$ and 192/768): the experiment measures the representation, not the input size. For wide clause and random 3-CNF the input is compact. Table 11 separates direct parity from parity encoded with auxiliaries; only the first has a one-piece ADSOP by construction, and the second must be measured on the encoded CNF, not assumed.

10.5 Limits and open problems

The alternatives do not cross the #P barrier. Exact counting remains #P-complete [14]; under #ETH (the counting variant of ETH) there is no $2^{o(n)}$ algorithm for #SAT; sampling and hashing give approximation, not exactness; Karp–Luby estimates $|U|$ but complement subtraction destroys relative accuracy in the rare regime. ADSOP is exact for affine families, but the prototype uses a SAT oracle and may be exponential. If there were a uniform polynomial ADSOP compiler for every CNF, it would follow that #SAT $\in$ FP and, by Toda, PH = P; the prototype is not that compiler.

Open Problem (ADSOP compilation)**.** Characterize the families of CNFs that admit an ADSOP of size poly(n, m) and decide whether there is a uniform polynomial compiler that builds it. Parity shows that the DSOP bound is not inherited; a random set suggests that the affine partition can also be exponential, but we do not know an explicit lower bound for a polynomial-size CNF family.

Open Problem (Compact parity)**.** Solve parity with a compact CNF (with auxiliary variables and existential counting) in the ADSOP prototype. The full expansion used in the experiments has 2^{n-1} clauses; a compact CNF requires handling the projection onto the original variables.

Open Problem (Affine backdoor)**.** Find a small set B of variables such that, after conditioning on B, the residual is affine for every assignment of B. Then

$$\#\mathsf{SAT}(F) = \sum_{\beta \in \{0,1\}^{|B|}} \#\mathsf{SAT}(F|_\beta)$$

and each term is counted by linear algebra. The direct prototype builds a backdoor on the fly; the question is whether real CNFs have a small affine backdoor.

11 Experimental protocol

The following protocol makes the engineering claims falsifiable and allows the three branches of the engine to be compared.

11.1 Instances

Family	Parameters	What it measures
Wide clause	$n \leq 24$, $w = 2, \ldots, n$	$K = 1$, $O(n)$ queries
Parity	$n \leq 16$	$K = 2^{n-1}$, limit of the DSOP engine
Random 3-CNF	$n \leq 20$, $m = 4.2n$	typical K and time
PHP	p pigeons, $p - 1$ holes	cost of the unsatisfiability proof
Tseitin	3-regular graphs, odd charge	proof cost and trace size

Table 10: Families of the experimental protocol.

11.2 Metrics

For each instance we record: n, m, #SAT, popcnt(S), COVERTRACE's K, ZDD size (if built), certificate bytes, preprocessing time, time per bit, rank, and select query, and memory. For $n \leq 16$ every query is compared with brute force; for $n \leq 24$ at least the total count is compared with an independent implementation.

11.3 Acceptance criteria

(A1) For $n \leq 8$, every value of bit, rank, and select matches brute force.
(A2) For $n \leq 16$, #SAT matches the exact count by enumeration.
(A3) In the range $n \leq 16$, a corrupted DSOP certificate is rejected by the verifier; a valid certificate produces the same count as brute force.
(A4) The counting branch, when available, produces the same $\text{rank}_0(k)$ as the DSOP branch on instances where both are viable.
(A5) Uniform sampling passes a chi-square test for $n \leq 12$.

11.4 Experimental hypotheses

- Wide clauses produce small K and $O(n)$ queries; the balanced form would fragment them into 2^{n-w} singletons.
- Parity produces $K = 2^{n-1}$; the DSOP engine is exponential in n and the counting oracle (or a ZDD built from the linear equation) is the alternative.
- On random 3-CNF, K grows with n; the crossover between DSOP and counting depends on the instance and must be measured, not postulated.

11.5 Protocol for the alternatives

The alternatives of Section 10 are evaluated with two independent protocols. The stochastic one uses instances with known #SAT for $n \leq 26$: unique solution, disjoint clauses, random 3-CNF, and one UNSAT instance. We compare direct sampling with 120 000 samples, XOR hashing with a cap of 64 models and seven repetitions, and Karp–Luby with 50 000 samples. We record the estimate, the relative error, and the number of constraints t (Tables 6 and 7). The affine protocol uses $n \leq 10$: the direct ADSOP is compiled, each piece is validated against the full table, and the number of pieces, SAT calls, and nodes are compared with the brute affine cover and the heuristic DSOP (Table 9).

The acceptance criteria are:

(A6) Every direct ADSOP cover is disjoint and covers exactly $U(F)$ for $n \leq 10$.

(A7) The ADSOP construction does not use the full list of points; the generation of test CNFs and the final validation do.

(A8) Stochastic estimates are compared with the exact count and the error is reported, without claiming exactness.

(A9) The Karp–Luby experiment reproduces complement cancellation when $\#\mathsf{SAT}/2^n$ is tiny.

The hypotheses are: direct ADSOP compresses parity and cube+parity; it does not compress a random 3-CNF; the direct compiler may produce fewer pieces than greedy partitioning because the SAT hull detects large pieces, but it is not a polynomial compiler.

11.6 Protocol for the hybrid

The evaluation of the hybrid should not be limited to showing that one branch wins on a benchmark. Table 4 is the first structural step; it does not replace the measurement of times or scaling. For each family we record the witness or parameter that activates the branch, construction time, success or UNAVAILABLE, query time, and correctness against brute force. Table 11 fixes the minimum matrix.

Family	Generator	n	Branch / expected condition
Wide	one or few clauses of width $n/2$	10–40	DSOP with small K.
Direct parity	set of odd-parity assignments, no auxiliaries	6–20	One-piece ADSOP; exponential DSOP.
Encoded parity	compact CNF with auxiliaries (Tseitin/XOR)	6–20	True ADSOP of the encoded CNF; no one-piece assumption.
Cube+parity	expansion by assignment	8–12	ADSOP 2; exponential DSOP.
Random 3-CNF	$m = rn$, $r = 2, 3, 4, 4.2, 5$	10–30	Outside the conditions; counter or UNAVAILABLE.
Components	disjoint blocks	10–30	DSOP or counter per component.
Treewidth	chain and bounded-pathwidth variants	10–40	Counter, DSOP, or ZDD.
Affine backdoor	XOR/affine plus a small CNF	10–20	Affine counter or ADSOP.
Tseitin / PHP	standard generators	10–30	DSOP or counter; stress test.
Planted	3-CNF with planted solution	20–50	Counter or estimation.
Real	DIMACS/SATLIB	variable	Mixed; the chosen branch is reported.

Table 11: Minimum matrix of families for the hybrid evaluation. The condition column is an expectation, not a result.

The structural metrics are n, m, clause width, components, a treewidth estimate (for example, minimum elimination), DSOP K, ZDD size Z, spectrum sparsity s, number A of ADSOP pieces, and availability of counter A. For instances with auxiliaries we record separately the number of original variables, the total number of variables, the encoding used (Tseitin, native XOR, full expansion), and the exact region covered. The execution metrics are preprocessing time, time per query for bit, rank, select, $\#\mathsf{SAT}$, and sampling, memory, success rate, and UNAVAILABLE rate. For $n \leq 16$ every query is compared with brute force; for $n \leq 24$ the total count is compared with an independent implementation; for stochastic branches we report relative error and confidence interval.

The controls are: brute force for small n, `sharpSAT`, `c2d`, or `Dsharp` as independent counters and knowledge compilers, ApproxMC for estimation, COVERTRACE for DSOP, the ADSOP prototype for the affine branch, and a materialized bitvector with a succinct index for rank/select at $n \leq 16$. Each family is labeled *proved polynomial*, *conditional*, *empirical*, or UNAVAILABLE; a polynomial fit on a benchmark is not presented as a complexity bound. Times are reported as work and, if parallelism is measured, also as depth and work factor; wall-clock times are not

compared across different cost models.

The acceptance criterion is: exact correctness in exact branches, controlled error in stochastic ones, no branch claims success if it does not cover $U(F)$, and reproducibility of generators and seeds. The evaluation must show both the regimes where the engine is competitive and those it leaves out; an exponential result on one family does not refute another regime, and a polynomial result on a benchmark does not prove a uniform bound.

11.7 Three measurement scenarios

The protocol distinguishes three scenarios that Table 12 does not separate by itself. The first is the total time of a single query, including compilation: it is the relevant scenario when a formula is queried once. The second is the amortized time for Q queries on the same formula:

$$T_{\text{total}}(Q) = T_{\text{setup}} + Q\,T_{\text{query}}, \qquad Q^* = \frac{T_{\text{setup}}}{T_{\text{Kerberos}} - T_{\text{query}}},$$

where Q^* is the number of queries from which the compiled branch pays for its cost. The third is the time of rank and select against a baseline implementing the same operations, for example a materialized bitvector with a succinct index [12, 13] for $n \leq 16$, or a linear scan of $S(F)$ when materialization is not viable.

Table 12 is a single-run pilot and does not satisfy the full protocol. A publishable version requires at least ten seeds per configuration, mean and standard deviation, the version of each program, the hardware, the exact commands, and the generators. The comparison with Kerberos/BASILISK is useful because it separates compilation cost from query cost, but it does not replace comparison with a standard knowledge compiler (c2d, Dsharp, or sharpSAT) or with a materialized rank/select index. Those comparisons are part of the protocol and remain pending. The pilot did not record hardware or seeds; it is therefore not presented as reproducible. It is a declared limitation, not a result to be taken as a definitive measurement.

ADSOP unit tests. In addition to coverage tests, the protocol includes four boundary cases for the corrected formula of Theorem 10.4: (i) an affine piece and a pattern v whose system $M_{j,v}$ is inconsistent, which must contribute 0; (ii) a zero-dimensional intersection, that is, a consistent system of rank d_j, which must contribute 1; (iii) a consistent intersection of intermediate rank, which must contribute $2^{d_j - r}$; and (iv) a zero-dimensional piece, which must contribute 1 if the point satisfies the pattern and 0 otherwise. Each case is compared with full enumeration for $n \leq 10$. Without these tests, an error such as the previous formula —assigning the full volume to an empty intersection— would go unnoticed in families where every intersection is consistent.

11.8 Preliminary results: DSOP versus Kerberos

The baseline of this evaluation is Kerberos: SLIME for SAT and BASILISK for #SAT. No other solvers are compared; the question is whether the paper's DSOP layer adds anything over the best engine available in the project. The comparison with c2d, Dsharp, or sharpSAT is specified in the protocol (Section 11.6) and remains pending; the conclusions of this pilot are therefore local to Kerberos and not an evaluation of the state of the art. We used `tools/bench_kerberos.cpp` for SLIME/BASILISK and `tools/paper_vs_kerberos.py` for the DSOP prototype. The times are from one run, single-threaded, with `-O3`; they do not include process startup or engine construction, and they are not a competition suite.

The DSOP prototype uses `brute_force` to enumerate $U(F)$ and `cover` to cover it; **it is not COVERTRACE**. Therefore the setup column measures a naive compiler, not the compiler the paper proposes. The comparison serves to separate the query layer from compilation cost.

Family	n	K	Setup (ms)	DSOP #SAT (μs)	Kerberos #SAT (ms)	rank (μs)	select (μs)
wide	12	1	1.89	0.70	0.068	7.2	35.8
parity	10	512	89.0	167.6	5.86	178.3	2644.8
parity	12	2048	1288.6	716.3	72.87	732.7	13422.5
cube+parity	10	512	104.5	173.6	7.49	177.0	3090.7
3-CNF $r = 2.0$	12	627	46.6	250.1	0.222	201.0	7701.1
3-CNF $r = 3.0$	12	134	34.0	56.8	0.147	51.5	1904.5
3-CNF $r = 4.2$	12	40	25.3	18.1	0.143	20.2	605.6
chain	12	1373	41.6	536.5	0.121	799.4	15045.7

Table 12: The paper's DSOP engine versus Kerberos/BASILISK at $n \leq 12$. The DSOP setup is the naive prototype (`brute_force+cover`), not COVERTRACE. The DSOP query is after setup; Kerberos #SAT is a BASILISK call.

Family	n	m	#SAT	Time (ms)	Nodes
wide	30	1	1 073 709 056	0.08	30
unique solution	20	20	1	0.04	1
disjoint	18	6	117 649	0.07	11
parity	12	2048	2 048	80.49	4095
3-CNF $r = 3.0$	24	72	1 584	1.06	390
3-CNF $r = 4.2$	24	100	0	0.25	13

Table 13: Kerberos/BASILISK: exact counting and wall-clock time. The wide, unique-solution, disjoint, and parity families have analytic values; the 3-CNFs were compared with an independent enumeration at $n \leq 24$.

The results separate two regimes. For a single #SAT query, Kerberos/BASILISK wins in every case: naive DSOP setup costs between 1.9 ms and 1.29 s, while BASILISK counts in 0.07–72.9 ms. After setup, the DSOP query is much cheaper: 0.7–716 μs versus 0.07–72.9 ms for BASILISK. For repeated queries, DSOP wins on wide, parity, and cube+parity; on random 3-CNF the advantage is small or nil, and on the chain Kerberos is faster. DSOP is the only branch that offers `rank` and `select`; Kerberos does not have them natively, and DSOP `select` costs between 36 μs and 15 ms depending on K and n.

The practical conclusion is that of the hybrid: use Kerberos/BASILISK for a point query or when the counter is efficient, and use DSOP when it is already compiled and many aggregate or rank/select queries will be made. The bottleneck remains the compiler: the naive prototype does not compete with COVERTRACE, which is precisely what remains to be implemented and measured. This comparison does not include the Anthropic algorithm, which has no practical implementation.

The break-even point Q^* is computed directly from the table. For the wide clause at $n = 12$, $Q^* \approx 28$ queries; for parity at $n = 10$ and $n = 12$, $Q^* \approx 16$ and $Q^* \approx 18$; for cube+parity at $n = 10$, $Q^* \approx 14$. In contrast, for 3-CNF with $r = 3.0$ at $n = 12$ about 377 queries are needed, and for $r = 4.2$ about 203. For $r = 2.0$ and the chain the denominator is negative: the DSOP query is slower than a Kerberos count, so the compiled branch never pays off in those cases. These figures are from a single run and serve to illustrate the calculation, not as a scaling conclusion; the full protocol requires the repetitions and baselines described in Section 11.7.

Test on CNFs emitted by Ariadna. The parameter `-dsop-count` was added to Ariadna. With it, `-analyze-cnf ... -count-cnf -dsop-count` uses the hybrid engine: materialized if $n \leq N_0$, DSOP if $n \leq N_{\text{dsop}}$ and $K \leq K_{\max}$, and BASILISK as fallback counter. Table 14 compares total wall-clock time with Ariadna's classical counter on CNFs emitted by `-emit-cnf` from Turing examples.

CNF	n	m	Ariadna (ms)	`-dsop-count` (ms)	Speedup	Branch
`n1`	88	252	28.9	29.0	1.00×	DSOP, $K = 1$
`n2`	1 531	4 560	343.3	317.8	1.08×	counter
`n3`	13 341	39 965	2 342.1	2 181.1	1.07×	counter
`n4`	83 593	250 695	22 134.6	23 281.1	0.95×	counter

Table 14: Total wall-clock time of `ariadna -analyze-cnf ... -count-cnf`, including analysis and parsing, on CNFs emitted by Ariadna. The `-dsop-count` column uses the automatic policy; n1 activates DSOP; n2–n4 exceed N_{dsop} and use BASILISK.

The result is not a uniform victory. In n1 DSOP detects a single cube ($K = 1$) and counts in microseconds, but total time is dominated by CNF analysis and does not improve. In n2 and n3 the BASILISK counter is 7–8% faster than the classical counter; in n4 it is 5% slower. Forcing DSOP in n2 takes 7.2 s in setup, far above BASILISK, so the automatic policy avoids it. The conclusion is the expected one: on Ariadna's large UNSAT CNFs, Kerberos/BASILISK is the practical engine; DSOP remains an opt-in branch for small/structured instances and repeated queries. Ariadna's default path is unchanged.

12 Related work

SAT Equation. The encoding of the truth table as an integer appears in [1] for balanced CNFs and in [2] for arbitrary CNFs via bitwise OR. The note [4] unifies this representation with COVERTRACE compilation and proves the fundamental identity between Hamming weight, the forbidden region, and the sum of volumes. This paper adds the query layer: bit, rank, select, #SAT, and sampling.

Knowledge compilation. DSOP is a form of knowledge compilation in the sense of [9]; in that literature's terminology it is a disjoint or partitioned DNF, and its study as a two-level form has its own history [26, 25]. COVERTRACE [3] provides a disjoint-subcube compiler with exact counting and witness extraction; the parity bound 2^{n-1} shows that cube compression has limits. Theorem 4.3 identifies DSOP with a DF-CNF, connecting compilation with clause syntax. The conditional implication of Theorem 7.1 is the same as Theorem 7.4 of [3]. Put plainly: the dense encoding is not the mathematical novelty of the paper; the novelty is the query layer, the correction of the affine case, and the barrier map.

Other knowledge forms. The knowledge-compilation literature contains d-DNNF, SDD, OBDD, and ZDD, with well-studied tractability hierarchies and compilation maps [9, 27, 28, 29, 30]. DSOP occupies a restricted place in that map: it is a two-level form, weaker than d-DNNF in expressiveness but with very direct counting and ranking queries. The honest comparison is not "DSOP versus d-DNNF" in general, but "DSOP with small K versus the best structure for the family". Proposition 4.8 shows that there are polynomial-size CNF families for which DSOP is not competitive; the question is whether some other form in the map is.

BDD and ZDD. Binary decision diagrams [10] and ZDDs [11] are canonical structures for functions and families of sets, given a variable order. Section 9 uses them as a query optimization, not as a substitute for DSOP.

Fourier and Walsh–Hadamard. The Walsh–Hadamard transform expresses the truth table in the character basis [8]. A sparse spectrum is another compressed representation: parity has two coefficients, while a wide clause has 2^w. The algorithm of [7] improves the dense DFT by

a polylogarithmic factor; for $N = 2^n$ it remains exponential in n. Compiling a sparse spectrum from the CNF is an open problem.

Rank and select. Succinct bitvectors allow rank and select in constant time with $2^n + o(2^n)$ bits [12, 13]. Later work improves space and time with Raman–Raman–Rao structures [31] and studies the limits of succinct queries with Pătraşcu–Viola [32]. That bound is excellent, but it requires materializing 2^n bits. Our DSOP engine avoids materialization at the cost of $O(nK)$ per query; the counting oracle avoids both at the cost of a call to an exact counter.

Model counting. Exact model counting is #P-complete in general [14]. The sparsification of [16] is one of the tools that reduces arbitrary CNFs to bounded-occurrence instances. The instantiation with [5, 6] is formalized as hypothesis H-ETH in Appendix A.

Approximate counting. The Karp–Luby FPRAS for unions of sets [20] and Stockmeyer's characterization of approximate counting in $\mathsf{BPP}^{\mathsf{NP}}$ [21] are the classical framework. For self-reducible problems, almost-uniform sampling and approximate counting are equivalent [22]; this explains why a uniform sample of solutions does not give the exact count. ApproxMC [23] instantiates XOR hashing with a SAT solver. Section 10.2 uses these ideas and shows complement cancellation: Karp–Luby estimates $|U|$, not #SAT.

Affine subspaces and compilation. ADSOP generalizes the cube DSOP to affine subspaces over $\mathbb{F}_2$; parity, the negative witness of DSOP, becomes a single piece. The idea sits in the Darwiche–Marquis knowledge-compilation map [9]: the representation language is changed to gain conciseness on affine families, at the cost of a harder compiler. The prototype of Section 10.4 uses a SAT solver and linear algebra; it is not a polynomial compiler.

Complexity. The NP-completeness of SAT is classical [17, 18]. The relation between the engine and P versus NP is analyzed in Section 7: the engine does not provide a uniform polynomial algorithm for #SAT, and the structures that would make it possible imply $\mathsf{PH} = \mathsf{P}$ by Toda's theorem [15].

Proof complexity. The size K of a DSOP can be read as the measure of a refutation in a disjoint-cube system. The comparison with resolution, *cutting planes*, and *polynomial calculus* relies on the classical literature [33, 34]. Proposition 4.8 is a lower bound for that system on a concrete family; a more general theory of lower bounds for DSOP and ADSOP is part of the program of Section 7.14.

13 Discussion and open problems

13.1 Unconditional results

The unconditional results are the exact equivalence between disjoint covers of $U(F)$ and DF-CNFs equivalent to F (Theorem 4.3); the rank–select engine with cost $O(nK)$ and $O(n^2K)$ (Theorem 5.3); the reduction from $\mathrm{rank}_0(k)$ to exact counting with the prefix comparator (Theorem 6.2); the correctness of the hybrid engine (Theorem 8.1); the exact queries on an ADSOP, with the correction of the inconsistent case (Theorem 10.4); and the separation of the DSOP route (Proposition 4.8). In addition, the validation of the direct ADSOP compiler of Section 10.4 constructs disjoint covers without enumerating the table for $n \leq 10$. The complexity frontier is also unconditional in its conditional direction: Theorem 7.1 proves that a uniform polynomial

DSOP compiler would imply $\mathsf{PH} = \mathsf{P}$, and Section 7 explains why the engine does not provide that compiler and which routes remain open.

The conceptual point is that the SAT Equation is not just a dense integer: its generalized form provides a query interface. DSOP compresses the forbidden region into cubes; the prefix comparator turns a range query into a counting instance; binary search turns counting into select and sampling.

13.2 The structural limit

The size K of the DSOP and the size Z of the ZDD are the parameters governing queries. Parity shows that K can be exponential in n; the parity ZDD is $O(n)$ because it uses a linear representation, but building it directly from the CNF is an open problem. The strong separation between polynomial-size CNF and polynomial-size DSOP is already established (Proposition 4.8); what is not proved here is the separation for ADSOP, ZDD, or Fourier. The engine does not eliminate the hardness of model counting: it shifts it to compilation or to the counter. ADSOP moves the limit from cubes to affine subspaces, but it does not eliminate compilation; stochastic oracles shift exactness to approximation. Section 7 sharpens this statement with the routes of Table 1, the islands of tractability, and the exact collapse condition.

13.3 Open problems

Open Problem (Cube compression)**.** Characterize the families of CNFs for which $\chi_{\sqcup}(U(F))$ is polynomial in n, m and, moreover, there is a polynomial compiler producing the DSOP. The existence of families with $m = \text{poly}(n)$ and $\chi_{\sqcup} = 2^{\Omega(n)}$ is no longer open: Proposition 4.8 establishes it. What remains open is the fine characterization and coverage verification.

Open Problem (Affine separation)**.** Prove or refute Conjecture 7.5: is there a family of CNFs with $m = \text{poly}(n)$ whose minimum ADSOP is $2^{\Omega(n)}$? Parity shows that ADSOP can compress what DSOP cannot; a lower bound for a polynomial-size family is missing. An affirmative answer would close the affine route as a universal route. A negative answer is not enough for $\mathsf{P} = \mathsf{NP}$: eliminating the exponential bound does not give a polynomial bound. The collapse would require Conjecture 7.4, which is stronger.

Open Problem (Minimum-size decision)**.** Determine the complexity of deciding whether $\chi_{\sqcup}(U(F)) \leq K$ for a CNF F and an integer K. Absolute DSOP minimization is NP-hard for functions given by truth tables [26]; the version with a CNF as input and K as a parameter is unresolved. A reduction from SAT to this decision would give a direct connection between cube compression and NP.

Open Problem (Direct ZDD compilation)**.** Characterize the families of CNFs that admit a polynomial-size ZDD and build it directly, without enumerating a DSOP. Parity is a witness case: its ZDD is $O(n)$, but its minimum DSOP is 2^{n-1}.

Open Problem (Fourier compression)**.** Characterize the families of CNFs whose bitvector $S(F)$ has a Walsh–Hadamard spectrum of polynomial size and build it in polynomial time. Parity has spectrum size 2 and DSOP 2^{n-1}; a wide clause of width w has spectrum 2^w and DSOP 1. If every CNF had a polynomial spectrum constructible in polynomial time, it would follow that $\#\mathsf{SAT} \in \mathsf{FP}$ and $\mathsf{P} = \mathsf{NP}$.

Open Problem (Query–space tradeoff)**.** Determine whether there is a structure of size $\text{poly}(n, m)$ that answers rank in $\text{polylog}(n)$ time for CNF families with exponential DSOP, or prove a conditional lower bound. If such a structure were built in polynomial time for every CNF, it would follow that $\#\mathsf{SAT} \in \mathsf{FP}$ and, by Toda, $\mathsf{PH} = \mathsf{P}$.

Open Problem (Oracle verification)**.** Formally verify, independently, the algorithms of [5, 6], or identify the exact point that prevents the subexponential bound. The framework of Theorem 6.2 separates that task from the query layer.

Open Problem (Coverage verification)**.** The problem was stated in Section 7.9: determine whether a DSOP of $U(F)$ can be verified in polynomial time with a construction trace, and characterize the families for which verification without a trace is polynomial.

13.4 Alternatives and guarantees

Sections 10.2 and 10.3 do not compete with the DSOP engine on the same axis. Stochastic oracles reduce estimation cost at the price of exactness: direct sampling fails in the rare regime, XOR hashing finds it but only approximates, and Karp–Luby estimates $|U|$ with an FPRAS that complement subtraction does not turn into an FPRAS for #SAT. ADSOP provides exactness for affine families: parity goes from 2^{n-1} cubes to one piece and cube+parity from $1+2^{n-2}$ cubes to two. The cost is a harder compiler: the direct prototype uses a SAT oracle, makes $O(n^2)$ queries per node, and may explore an exponential number of nodes. The experiment for $n \leq 10$ shows that direct construction is viable and that on random 3-CNF it produces fewer pieces than brute greedy partitioning, but it is not a polynomial compiler. In both cases the representation changes and the cost shifts; no polynomial algorithm for #SAT is obtained.

13.5 Relation to practice

Slime and the practical comparison. The Slime solver of the Ariadna repository is a CDCL engine with HESS, CT, and the Basilisk counting backend; it is the practical base for SAT queries and the counter branch. The Anthropic algorithm is not a CDCL solver, publishes no implementation or benchmarks, and has astronomical constants; it does not compete with Slime in the experimental regime. The useful comparison is Slime/CDCL versus the portfolio, not the asymptotic algorithm. For a fast solver, the leverage is in the CDCL engine, preprocessing, branch selection, and counter integration; the Anthropic result is a theoretical reference, not a production implementation.

The hybrid engine suggests a concrete architecture for Ariadna and `satx`: a representation layer with a certificate verifier with traces, a rank–select engine over DSOP, and an adapter for exact counters. The function bit is always answered by evaluation; aggregate queries choose the representation. This separation ensures that an error in the counter does not affect point queries and that a DSOP certificate is verified independently. For small n, coverage is verified by enumeration; for larger n, the COVERTRACE trace is part of the certificate. The layer may include an ADSOP and a stochastic estimator, but the verifier must distinguish an exact answer from an approximation with a confidence interval; that distinction is part of the interface contract.

14 Conclusions

The SAT Equation encodes the truth table in a 2^n-bit integer. Point queries on that integer are direct; aggregate queries are the real problem. This paper shows that the generalized form, together with COVERTRACE compilation, yields a rank–select engine whose cost depends on the DSOP size, and that an exact model counter yields a rank oracle through the prefix restriction. The combination is a hybrid engine that avoids materializing the table and uses each representation in its competitive regime.

The strongest result remains unconditional: the DSOP–DF-CNF equivalence, the associated query engine, and the reduction from rank to counting. Two results that change the map are

added. The first is the correction of the ADSOP theorem: an inconsistent intersection contributes zero, and the prefix algorithm must handle that case; without the correction, rank and select counts on an ADSOP were wrong. The second is the separation of the DSOP route: the Sasao construction, discussed by Bernasconi, Ciriani, Luccio, and Pagli, gives a family of CNFs with $O(n)$ clauses and minimum DSOP $2^{\Omega(n)}$. This does not prove $\mathsf{P} \neq \mathsf{NP}$, but it rules out disjoint cubes as a universal representation and turns the program into a question about the other islands. ADSOP keeps parity in one piece and answers exact queries; the direct prototype validates the representation for $n \leq 10$, but it is not a polynomial compiler and its lower bound remains open. ZDD and Fourier are presented as conditional, not universal, compressions. The subexponential part (Appendix A) is conditional on H-ETH, announced by two manuscripts but not verified, and is declared as such. The preliminary evaluation against Kerberos/BASILISK shows that Kerberos wins on a single #SAT query and that the DSOP prototype wins on repeated queries and `rank`/`select` once compiled; the naive compiler remains the bottleneck. On Ariadna's large CNFs the improvement is not uniform (7–8% on n2/n3, −5% on n4), so `-dsop-count` remains an option and the default path is unchanged. Section 7 answers the complexity question: the engine does not imply $\mathsf{P} = \mathsf{NP}$ because its polynomial bounds are conditional on the certificate or the counter. But the answer is not only negative: the paper states the islands of tractability, proves that the DSOP island does not cover every CNF, and leaves the affine route bounded by two extremes, universality and separation, without claiming that they are exhaustive. Parallelization does not change the diagnosis: total work, not depth, is the decisive resource. The open agenda is now more precise: lower bounds for ADSOP, affine backdoors, direct ZDD and Fourier compilation, certificate verification, and proof complexity.

Limitations. This manuscript does not prove $\mathsf{P} = \mathsf{NP}$ or $\mathsf{P} \neq \mathsf{NP}$. The DSOP lower bound relies on the Sasao construction and the discussion by Bernasconi, Ciriani, Luccio, and Pagli; we do not reproduce it here. The ADSOP compiler is a prototype with a SAT oracle, not a polynomial compiler. The experimental evaluation is a single-run pilot. The H-ETH part depends on unverified manuscripts and is relegated to Appendix A. The program toward $\mathsf{P} = \mathsf{NP}$ is a research agenda, not a result. The value of the paper, if any, lies in the correction of the affine case, the separation of the DSOP route, and the map of what is missing; not in a solution to the millennium problem.

A The H-ETH instantiation

A conditional note. This note may be skipped on a first reading: it does not affect the results of Sections 4–5 or the barrier map. It is included because the 2026 manuscripts are the only available subexponential instantiation and because it is worth separating what is known from what is announced.

Hypothesis A.1 (H-ETH)**.** There is a deterministic algorithm that, for each fixed k, counts the satisfying assignments of a k-CNF with n variables and m clauses in time $2^{o(n)}\,\mathrm{poly}(n, m)$ and polynomial space. The OpenAI [5] and Anthropic [6] manuscripts announce subexponential algorithms for SAT; H-ETH is the counting form needed in this paper. H-ETH implies that ETH is false.

H-ETH is stronger than the negation of ETH for decision: it is the property required by the following corollary. If a manuscript only refuted ETH for decision, H-ETH would remain an additional hypothesis and the corollary would be read under that hypothesis.

Observation A.2 (Status of the evidence for H-ETH)**.** On October 26, 2026, OpenAI publicly announced a Lean formalization of a refutation of ETH and an external review process led by complexity specialists [5]. That announcement is not equivalent to community acceptance, and

the distinction matters: refuting ETH is not proving $\mathsf{P} = \mathsf{NP}$, because a $2^{o(n)}$ algorithm can still be superpolynomial. The Anthropic manuscript [6] claims a deterministic $2^{o(n)}$ algorithm for $\#k$-SAT and a Lean 4 formalization; if correct, it refutes ETH and SETH and gives a subexponential-time counter. It does not declare a space bound, and H-ETH requires polynomial space: without that additional analysis it does not establish H-ETH in the form of Corollary A.3. The claim is extraordinary and has not been verified by humans: the manuscript appendix states that the proof was written and reviewed by models, that the Lean listing contains definitions and the complete proof is in a separate repository, and that the constant is not explicit. The manuscript itself notes that it offers no practical algorithms and that its improvement does not appear until $n > 10^{10\,000}$ in the case $m = 5n$. We do not use the claim as established: Corollary A.3 remains conditional on H-ETH. If it is confirmed and the counter uses polynomial space, the corollary becomes unconditional for k-CNF, but no polynomial or practical algorithm is obtained.

Corollary A.3 (Under H-ETH)**.** *If H-ETH holds, then for every k-CNF F (of fixed width k) and every prefix $0 \le t \le 2^n$:*

(a) $\mathrm{rank}_0(t)$ *is computed in* $2^{o(n)}\,\mathrm{poly}(n, m)$ *time and polynomial space.*

(b) $\mathrm{select}_0(r)$ *and uniform sampling are computed in* $2^{o(n)}\,\mathrm{poly}(n, m)$ *time and polynomial space.*

Proof. Apply Theorem 6.2 with the H-ETH counter. The comparator adds $O(n)$ variables and $O(n)$ clauses; the combined CNF has bounded width (at most $\max(k, 3)$) if it is encoded with bounded-fan-in gates and Tseitin, and H-ETH applies to that fixed width. The number of queries for select_0 is $O(n)$, and $n \cdot 2^{o(n)} = 2^{o(n)}$. □

Observation A.4 (Arbitrary width)**.** H-ETH is formulated for k-CNF. For a CNF of arbitrary width one must pay for an encoding to k-CNF (Tseitin or sparsification); the resulting bound is in the number of encoded variables, not necessarily in the original n. The comparator reduction is parsimonious in the count but not in width: it preserves the number of models, not the width of the clauses of F. That is why Corollary A.3 is stated for k-CNF and not for arbitrary CNFs. For a CNF with unbounded-width clauses, the right route is DSOP or a representation that does not pay the encoding. The DSOP engine of Section 5 does not have this width restriction; hence the two methods are complementary.

References

[1] O. Riveros. *A Unique Encoding of Satisfying Assignments for Balanced CNFs.* Manuscript, 2025. Included in *Complete Works*, Vol. I.

[2] O. Riveros. *A Generalization of the SAT Equation Theorem to Arbitrary CNF Formulas via Bitwise OR Encoding.* Manuscript, 2025. Included in *Complete Works*, Vol. I.

[3] O. Riveros. *COVERTRACE-SAT as Disjoint-Subcube Knowledge Compilation.* Unified document of Vol. I, 2026. Included in *Complete Works*, Vol. I.

[4] O. Riveros. *Connections between the SAT Equation Theorem and COVERTRACE-SAT.* Unifying note, 2026. Included in *Complete Works*, Vol. I.

[5] OpenAI. *The Exponential Time Hypothesis is False.* Manuscript, October 17, 2026 (revised October 25, 2026). `https://github.com/openai/math/blob/main/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf` Accessed October 27, 2026.

[6] Claude and Anthropic. *A $2^{o(n)}$ time algorithm for 3-SAT.* Manuscript, October 26, 2026.

[7] OpenAI. *An explicit power saving for the exact discrete Fourier transform.* Manuscript, September 25, 2026.

[8] R. O'Donnell. *Analysis of Boolean Functions.* Cambridge University Press, 2014.

[9] A. Darwiche and P. Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002.

[10] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, 35(8):677–691, 1986.

[11] S. Minato. Zero-suppressed BDDs for set manipulation in combinatorial problems. In *Proceedings of the 30th Design Automation Conference*, pages 272–277, 1993.

[12] G. Jacobson. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science*, pages 549–554, 1989.

[13] G. Navarro and E. Providel. Fast, small, simple rank/select on bitmaps. In *Proceedings of the 11th International Symposium on Experimental Algorithms*, pages 295–306, 2012.

[14] L. G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979.

[15] S. Toda. PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991.

[16] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001.

[17] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.

[18] L. A. Levin. Universal sequential search problems. *Problemy Peredachi Informatsii*, 9(3):115–116, 1973.

[19] S. Arora and B. Barak. *Computational Complexity: A Modern Approach.* Cambridge University Press, 2009.

[20] R. M. Karp and M. Luby. Monte-Carlo algorithms for enumeration and reliability problems. In *Proceedings of the 24th Annual Symposium on Foundations of Computer Science*, pages 56–64, 1983.

[21] L. Stockmeyer. The complexity of approximate counting. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, pages 118–126, 1983.

[22] M. Jerrum, L. Valiant, and V. Vazirani. Random generation of combinatorial structures from a uniform distribution. *Theoretical Computer Science*, 43:169–188, 1986.

[23] S. Chakraborty, K. S. Meel, and M. Y. Vardi. A scalable approximate model counter. In *Principles and Practice of Constraint Programming*, pages 200–216, 2013.

[24] M. Mézard, G. Parisi, and R. Zecchina. Analytic and algorithmic solution of random satisfiability problems. *Science*, 297:812–815, 2002.

[25] T. Sasao. A design method for AND-OR-EXOR three-level networks. In *Proceedings of the ACM/IEEE International Workshop on Logic Synthesis*, pages 8:11–8:20, 1995.

[26] A. Bernasconi, V. Ciriani, F. Luccio, and L. Pagli. Compact DSOP and partial DSOP forms. *Theory of Computing Systems*, 53(4):583–608, 2013.

[27] A. Darwiche. Decomposable negation normal form. *Journal of the ACM*, 48(4):608–647, 2001.

[28] A. Darwiche. SDD: A new canonical representation of propositional knowledge bases. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence*, pages 819–826, 2011.

[29] J. Huang and A. Darwiche. DPLL with a trace: from SAT to knowledge compilation. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence*, pages 156–162, 2005.

[30] T. Pipatsrisawat and A. Darwiche. A lower bound on the size of decomposable negation normal form. In *Proceedings of the 24th AAAI Conference on Artificial Intelligence*, pages 345–350, 2010.

[31] R. Raman, V. Raman, and S. S. Rao. Succinct indexable dictionaries with applications to encoding k-ary trees and multisets. In *Proceedings of the 13th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 233–242, 2002.

[32] M. Pătraşcu and E. Viola. Cell-probe lower bounds for succinct partial sums. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 117–122, 2010.

[33] E. Ben-Sasson and A. Wigderson. Short proofs are narrow—resolution made simple. *Journal of the ACM*, 48(2):149–169, 2001.

[34] A. A. Razborov. Lower bounds on the monotone complexity of some Boolean functions. *Doklady Akademii Nauk SSSR*, 281:798–801, 1985.